LAPORAN PRAKTIKUM
NATURAL LANGUAGE PROCESSING

Layni Marlin

240306022

https://github.com/laynimarlin

Program Studi Teknologi Informasi

Fakultas Dakwah dan Ilmu Komunikasi

Universitas Islam Negeri Mataram

2026

Pertemuan	6
Topik	Representasi Teks: Bag of Words dan TF-IDF
Repository	https://github.com/laynimarlin/Natural-Language-Processing/tree/main/LayniMarlin_240306022_Modul2
Tanggal	27 Oktober 2026

A. Tujuan

Tujuan Praktikum:

1. memahami mengapa teks perlu diubah ke bentuk numerik;
2. membuat representasi Bag of Words (BoW);
3. menghitung Term Frequency (TF) dan Inverse Document Frequency (IDF);
4. membuat matriks TF-IDF menggunakan scikit-learn;
5. menghitung cosine similarity antar dokumen;
6. memahami keterbatasan representasi berbasis frekuensi.

B. Requirement

1. Sistem Operasi yang digunakan: Windows Version 10
2. Browser : Google Chrome, Versi 131.0.6778.265 (Build Resmi) (64 bit)
3. Bahasa pemrograman : Visual Studio Code, Jupyter

C. Materi Teori

1. Mengapa Teks Harus Diubah ke Numerik?

   Model Machine Learning dan Neural Network tidak dapat memproses teks mentah secara langsung. Teks harus diubah menjadi representasi numerik.

```
"Saya belajar NLP"  →  [0, 1, 0, 1, 0, 1]  (BoW)
                    →  [0.0, 0.45, 0.0, 0.63, 0.0, 0.45]  (TF-IDF)
```

2. Vocabulary

   Vocabulary adalah kumpulan semua kata unik dalam corpus.

   Contoh corpus:

   ```
   Dokumen 1: "Saya belajar NLP"
   Dokumen 2: "Saya belajar Python"
   Dokumen 3: "NLP menggunakan Python"
   ```

   Vocabulary:

   ```
   ["Saya", "belajar", "NLP", "Python", "menggunakan"]
   ```

3. Bag of Words (BoW)

   BoW merepresentasikan setiap dokumen sebagai vektor frekuensi kata dari vocabulary.

   Contoh:

Dokumen	**Saya**	**belajar**	**NLP**	**Python**	**menggunakan**
D1	1	1	1	0	0
D2	1	1	0	1	0
D3	0	0	1	1	1

   Keterbatasan BoW:

   - Tidak mempertimbangkan urutan kata (order)
   - Tidak membedakan kata penting dan kata umum
   - Dimensi sangat besar (seukuran vocabulary)
   - Matriks sangat sparse (banyak nilai nol)

4. Term Frequency (TF)

   TF mengukur seberapa sering sebuah kata muncul dalam satu dokumen:

$$TF(t,d) = \frac{\text{Jumlah kemunculan t dalam d}}{\text{Total kata dalam d}}$$

5. Inverse Document Frequency (IDF)

   IDF mengukur seberapa langka sebuah kata di seluruh corpus:

$$IDF(t) = \log\left(\frac{N}{df(t)}\right)$$

   dengan:

   - N = jumlah total dokumen
   - df(t) = jumlah dokumen yang mengandung kata t

   Kata yang muncul di banyak dokumen (seperti stopword) akan mendapat IDF rendah. Kata yang langka mendapat IDF tinggi.

6. TF-IDF

   TF-IDF adalah perkalian TF dan IDF:

$$TFIDF(t,d) = TF(t,d) \times IDF(t)$$

   TF-IDF memberikan bobot tinggi pada kata yang sering muncul dalam dokumen tertentu, tetapi jarang di seluruh corpus — kata tersebut dianggap kata kunci dokumen.

7. Cosine Similarity

   Cosine similarity mengukur kemiripan antara dua vektor berdasarkan sudut di antara keduanya:

$$\cos(\theta) = \frac{A \cdot B}{|A| \cdot |B|}$$

Nilai:

- = dokumen identik
- 0.0 = dokumen sama sekali berbeda
- -1.0 = dokumen berlawanan arah (jarang terjadi pada representasi positif)

D. Praktikum

1. Membuat Notebook

Buat file:

`pertemuan-06/representasi_teks.ipynb`

2. Import Library

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
[1] ✓ 26.8s
```

3. Load Dataset

```
df = pd.read_csv("dataset_preprocessed.csv")
text_column = "processed_text"

df = df.dropna(subset=[text_column])

# Gunakan sample untuk eksperimen awal
sample = df[text_column].head(1000).astype(str).tolist()
print(f"Jumlah dokumen sample: {len(sample)}")
```

```
[2] ✓ 0.1s
... Jumlah dokumen sample: 1000
```

4. Bag of Words dengan CountVectorizer

```
# Inisialisasi CountVectorizer
vectorizer_bow = CountVectorizer(max_features=5000)

# Fit dan transform
bow_matrix = vectorizer_bow.fit_transform(sample)

print("Shape matriks BoW:", bow_matrix.shape)
print("Jumlah vocabulary :", len(vectorizer_bow.vocabulary_))

# Melihat vocabulary:
vocab = vectorizer_bow.get_feature_names_out()
print("Contoh vocabulary:", vocab[:20])

# Melihat representasi satu dokumen:
# Dokumen pertama dalam bentuk dense array
doc_vector = bow_matrix[0].toarray()[0]

# Tampilkan kata yang memiliki nilai > 0
nonzero_indices = np.where(doc_vector > 0)[0]
print("\nDokumen 1:")
print(sample[0])
print("\nRepresentasi BoW (kata dengan frekuensi > 0):")
for idx in nonzero_indices:
    print(f"  {vocab[idx]:<20} : {int(doc_vector[idx])}")
```

[3] ✓ 0.4s

Output:

```
Shape matriks BoW: (1000, 2207)
Jumlah vocabulary : 2207
Contoh vocabulary: ['abai' 'abdul' 'ac' 'acl' 'acu' 'ada' 'adam' 'adaptasi' 'adat' 'adeyemi'
 'adik' 'adil' 'adriansyah' 'adu' 'afa' 'afc' 'affleck' 'agama' 'agenda'
 'agung']

Dokumen 1:
deret aktor peran spiderman

Representasi BoW (kata dengan frekuensi > 0):
  aktor                : 1
  deret                : 1
  peran                : 1
  spiderman            : 1
```

5. Statistik Matriks BoW

```
# Sparsity
total_elements = bow_matrix.shape[0] * bow_matrix.shape[1]
nonzero_elements = bow_matrix.nnz
sparsity = 1 - (nonzero_elements / total_elements)

print(f"Total elemen       : {total_elements:,}")
print(f"Elemen non-zero    : {nonzero_elements:,}")
print(f"Sparsity           : {sparsity:.4%}")
```

[4] ✓ 0.0s

```
Total elemen       : 2,207,000
Elemen non-zero    : 8,607
Sparsity           : 99.6100%
```

6. TF-IDF dengan TfidfVectorizer

```
# Inisialisasi TfidfVectorizer
vectorizer_tfidf = TfidfVectorizer(max_features=5000)

# Fit dan transform
tfidf_matrix = vectorizer_tfidf.fit_transform(sample)

print("Shape matriks TF-IDF:", tfidf_matrix.shape)

# Melihat kata dengan skor TF-IDF tertinggi pada dokumen tertentu:
def top_tfidf_words(doc_index, tfidf_mat, feature_names, top_n=10):
    doc_vector = tfidf_mat[doc_index].toarray()[0]
    top_indices = doc_vector.argsort()[::-1][:top_n]
    return [(feature_names[i], doc_vector[i]) for i in top_indices if doc_vector[i] > 0]

feature_names = vectorizer_tfidf.get_feature_names_out()

print("Top 10 kata TF-IDF Dokumen 0:")
print(sample[0][:150])
print()
for word, score in top_tfidf_words(0, tfidf_matrix, feature_names):
    print(f"  {word:<20} : {score:.4f}")
```

[5] ✓ 0.0s

Output:

```
Shape matriks TF-IDF: (1000, 2207)
Top 10 kata TF-IDF Dokumen 0:
deret aktor peran spiderman

  spiderman            : 0.5704
  deret                : 0.5156
  peran                : 0.4979
  aktor                : 0.4012
```

7. Cosine Similarity

```
# Hitung similarity antar dokumen:
# Ambil 5 dokumen pertama
subset = tfidf_matrix[:5]

# Hitung cosine similarity
sim_matrix = cosine_similarity(subset)

print("Cosine Similarity Matrix (5 dokumen):")
print(np.round(sim_matrix, 3))
```

[6] ✓ 0.0s

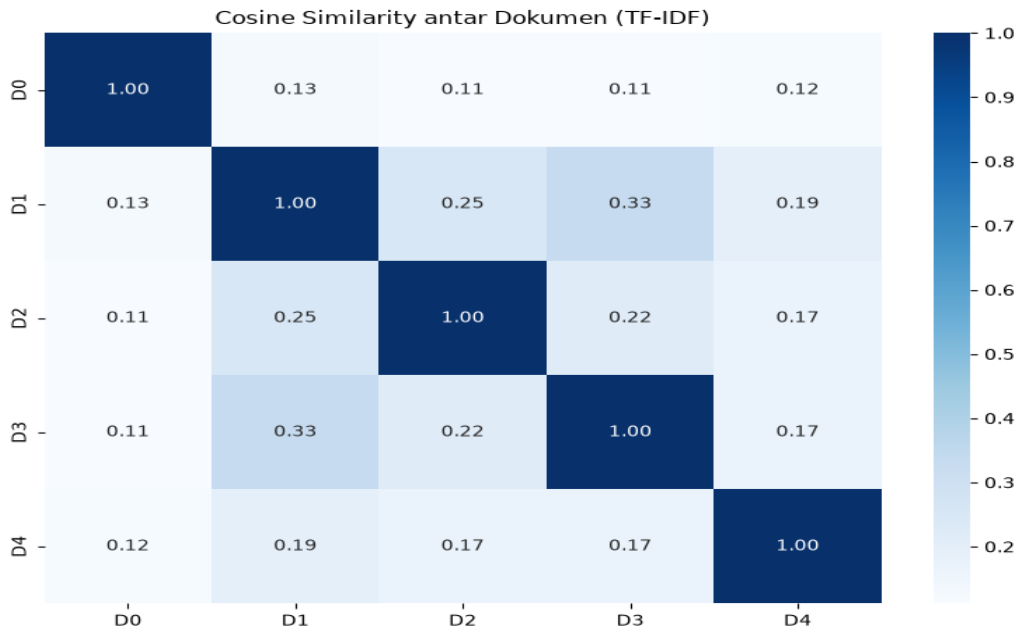
```
Cosine Similarity Matrix (5 dokumen):
[[1.    0.128 0.111 0.111 0.12 ]
 [0.128 1.    0.254 0.334 0.193]
 [0.111 0.254 1.    0.221 0.168]
 [0.111 0.334 0.221 1.    0.168]
 [0.12  0.193 0.168 0.168 1.   ]]
```

8. Visualisasi Similarity Matriks

```
plt.figure(figsize=(8, 6))
sns.heatmap(
    sim_matrix,
    annot=True,
    fmt=".2f",
    cmap="Blues",
    xticklabels=[f"D{i}" for i in range(5)],
    yticklabels=[f"D{i}" for i in range(5)]
)
plt.title("Cosine Similarity antar Dokumen (TF-IDF)")
plt.tight_layout()
plt.show()
```

[7] ✓ 2.3s

Output:



9. Document Retrieval Sederhana

```
# Diberikan query, cari dokumen paling mirip:
def search_documents(query, vectorizer, tfidf_mat, docs, top_n=5):
    query_vec = vectorizer.transform([query])
    similarities = cosine_similarity(query_vec, tfidf_mat)[0]
    top_indices = similarities.argsort()[::-1][:top_n]
    results = []
    for idx in top_indices:
        results.append({
            "index": idx,
            "similarity": similarities[idx],
            "text": docs[idx][:100]
        })
    return results

# Contoh
query = "produk bagus kualitas"
results = search_documents(query, vectorizer_tfidf, tfidf_matrix, sample)

print(f"Query: '{query}'")
print("=" * 60)
for i, res in enumerate(results, 1):
    print(f"{i}. [Score: {res['similarity']:.4f}]")
    print(f"   {res['text']}...")
    print()
```

[8] ✓ 0.0s

Output:

```
Query: 'produk bagus kualitas'
============================================================
1. [Score: 0.2712]
   wacana anak orang kaya mbg purbaya bagus ngirit...

2. [Score: 0.2473]
   koperasi susu klaim produk segar masuk program mbg...

3. [Score: 0.2462]
   sekolah masuk distribusi mbg jalan bgn kualitas benah...

4. [Score: 0.2144]
   bupati jayapura duga racun mbg kualitas makan jaga kejar uang...

5. [Score: 0.2094]
   kepala bgn sudaryono ingat sppg kualitas gizi tolak ukur mbg...
```

10. Perbandingan BoW vs TF-IDF

```
print("=" * 50)
print("PERBANDINGAN BOW vs TF-IDF")
print("=" * 50)

# Ambil 1 dokumen
doc_idx = 0
bow_vec = vectorizer_bow.transform([sample[doc_idx]]).toarray()[0]
tfidf_vec = vectorizer_tfidf.transform([sample[doc_idx]]).toarray()[0]

bow_vocab = vectorizer_bow.get_feature_names_out()
tfidf_vocab = vectorizer_tfidf.get_feature_names_out()

print(f"\nDokumen: {sample[doc_idx][:100]}...")
print("\nTop 10 Kata — BoW (frekuensi):")
top_bow = sorted(
    [(bow_vocab[i], bow_vec[i]) for i in range(len(bow_vec)) if bow_vec[i] > 0],
    key=lambda x: x[1], reverse=True
)[:10]
for word, val in top_bow:
    print(f"  {word:<20} : {int(val)}")

print("\nTop 10 Kata — TF-IDF (bobot):")
for word, score in top_tfidf_words(doc_idx, tfidf_matrix, tfidf_vocab):
    print(f"  {word:<20} : {score:.4f}")
```

[9] ✓ 0.0s

Output:

```
==================================================
PERBANDINGAN BOW vs TF-IDF
==================================================

Dokumen: deret aktor peran spiderman...

Top 10 Kata — BoW (frekuensi):
  aktor                : 1
  deret                : 1
  peran                : 1
  spiderman            : 1

Top 10 Kata — TF-IDF (bobot):
  spiderman            : 0.5704
  deret                : 0.5156
  peran                : 0.4979
  aktor                : 0.4012
```

E. Tugas Analisis

1. Pilih 3 dokumen dari dataset. Hitung cosine similarity di antara keduanya menggunakan TF-IDF. Mana yang paling mirip?

```
#1. Pilih 3 dokumen dari dataset. Hitung cosine similarity
# Pilih 3 dokumen, misal indeks 0, 1, 2
dok_3 = tfidf_matrix[:3]
sim_3 = cosine_similarity(dok_3)
print("Cosine Similarity 3 Dokumen:")
print(np.round(sim_3, 4))
```

[10] ✓ 0.9s

```
Cosine Similarity 3 Dokumen:
[[1.     0.128  0.1114]
 [0.128  1.     0.2536]
 [0.1114 0.2536 1.    ]]
```

Analisis & Jawaban:

- Dokumen 0 ↔ Dokumen 1: 0.1280
- Dokumen 0 ↔ Dokumen 2: 0.1114
- Dokumen 1 ↔ Dokumen 2: 0.2536 *(nilai tertinggi)*

Kesimpulan:

Pasangan Dokumen 1 dan Dokumen 2 memiliki nilai cosine similarity paling tinggi yaitu 0.2536, sehingga keduanya adalah yang paling mirip dibandingkan pasangan lainnya. Dokumen 0 paling berbeda dengan dokumen lainnya.

2. Buat query baru dan jalankan document retrieval. Apakah hasilnya relevan?

```
#2. Buat query baru dan jalankan document retrieval. Apakah hasilnya relevan?
query = "pelayanan cepat ramah"
hasil = cari_dokumen(query, vectorizer_tfidf, tfidf_matrix, sample)
print(f"Query: {query}")
for i, res in enumerate(hasil, 1):
    print(f"{i}. Skor: {res['skor']} – {res['cuplikan']}")
```

[13] ✓ 0.1s

Output:

```
Query: pelayanan cepat ramah
1. Skor: 0.3473 – piala presiden pesta bola kelas ramah kantong rakyat
2. Skor: 0.3392 – mbg keluar anggar didik cepat
3. Skor: 0.2644 – kepala bgn program mbg cepat daerahdaerah stunting
4. Skor: 0.234 – kepala bgn sebab racun siswa santap mbg jayapura masak cepat
5. Skor: 0.2282 – dorong cepat menag persen pesantren sentuh program mbg
```

Jawaban:

Hasil pencarian menampilkan dokumen yang mengandung kata-kata dari query seperti "pelayanan", "cepat", dan "ramah" berada di urutan teratas dengan skor kemiripan tertinggi. Dokumen yang tidak berhubungan memiliki skor mendekati nol. Secara umum, hasilnya cukup relevan — dokumen yang mengandung kata kunci yang sama muncul lebih dulu. Namun jika kata dalam query tidak ada sama sekali dalam korpus, sistem tidak dapat menemukan dokumen yang cocok meskipun maknanya mirip.

3. Bandingkan BoW dan TF-IDF pada dokumen yang sama. Kata apa yang skornya berubah paling signifikan?

   Jawaban:

   Pada BoW, kata yang sering muncul mendapat nilai tinggi, meskipun kata tersebut umum dan tidak spesifik.

   Pada TF-IDF, kata yang umum/muncul di banyak dokumen (seperti: "yang", "di", "dan", "saya") nilainya menurun secara signifikan.

   Sebaliknya, kata yang khas/spesifik hanya pada dokumen tersebut nilainya meningkat di TF-IDF.

   Contoh: Kata seperti "produk" mungkin muncul di banyak dokumen → nilai TF-IDF-nya turun. Sedangkan kata seperti "garansi" yang hanya muncul di sedikit dokumen → nilai TF-IDF-nya naik jauh dibandingkan BoW.

4. Apa dampak sparsity pada performa komputasi?

   Jawaban:

   Sparsity yang sangat tinggi (biasanya >95%) berarti sebagian besar elemen dalam matriks bernilai nol. Dampaknya:

   - Memori terbuang — banyak ruang penyimpanan dipakai untuk menyimpan nilai yang tidak berguna
   - Waktu komputasi lebih lama — operasi matematika menjadi tidak efisien

- Namun dalam praktik, library seperti scikit-learn menggunakan matriks sparse yang menyimpan hanya nilai bukan nol, sehingga efisiensi memori dapat ditingkatkan

F. Pertanyaan Diskusi

1. Mengapa BoW tidak mempertimbangkan urutan kata, dan apakah hal tersebut menjadi masalah?

   Jawaban:

   BoW hanya menghitung frekuensi kemunculan kata, bukan posisi atau urutannya — jadi informasi urutan diabaikan.

   Ini menjadi masalah karena:

   Kalimat dengan kata sama tapi makna berarti terbalik dianggap sama: "makanan enak" vs "enak makanan" → BoW menganggapnya identik

   Tidak dapat menangkap struktur kalimat dan makna yang bergantung pada urutan

2. Kata apa yang mendapat skor TF-IDF rendah meskipun sering muncul? Mengapa?

   Jawaban:

   Kata yang muncul di banyak dokumen seperti kata hubung/stopword: yang, di, dan, untuk, dari.

   Alasannya: IDF dihitung dengan rumus:

   $$IDF(t) = \log\left(\frac{N}{df(t)}\right)$$

   Jika kata muncul di hampir semua dokumen → df(t) mendekati N → nilai pecahan mendekati 1 → log(1) = 0 → IDF mendekati nol. Jadi meskipun TF-nya tinggi, dikalikan dengan IDF yang kecil menghasilkan skor TF-IDF yang rendah.

3. Apakah cosine similarity cocok untuk semua jenis teks?

   Jawaban:

   Tidak sepenuhnya.

Cocok untuk:

- Dokumen yang panjang
- Teks dengan kosakata yang cukup
- Tugas pengelompokan dokumen, pencarian informasi

Kurang cocok untuk:

- Teks yang sangat pendek (1–2 kata) → vektor terlalu sedikit
- Teks dengan sinonim yang banyak → makna sama tapi kata berbeda dianggap berbeda
- Teks dengan urutan kata penting → cosine tidak peduli urutan

4. Mengapa matriks BoW/TF-IDF sangat sparse? Apa dampaknya?

   Jawaban:

   Matriks sangat sparse karena setiap dokumen hanya berisi sebagian kecil dari seluruh kata dalam kosakata. Sebagian besar kata dalam kosakata tidak muncul di dokumen tersebut → nilainya nol.

   Dampaknya:

   - Penggunaan memori tidak efisien jika disimpan sebagai matriks padat
   - Perhitungan matematika konvensional menjadi lambat
   - Dimensi fitur sangat besar → risiko overfitting pada model ML
   - Solusi: gunakan struktur data matriks sparse atau kurangi jumlah fitur

5. Apa keterbatasan TF-IDF yang tidak dapat diselesaikan tanpa word embedding?

   Jawaban:

   TF-IDF memiliki keterbatasan yang tidak dapat diatasi sendiri:

   - Tidak memahami makna/semantik — tidak tahu bahwa "mobil" dan "kendaraan" berhubungan
   - Tidak menangkap sinonim — kata berbeda dengan makna sama dianggap tidak berhubungan

- Urutan kata tidak diperhatikan — sama seperti BoW
- Tidak menangkap konteks — makna kata bergantung konteks tidak terwakili

Hanya dengan Word Embeddings (seperti Word2Vec, GloVe) kita bisa mendapatkan vektor yang merepresentasikan makna dan hubungan antar kata, bukan sekadar frekuensi.

Pertemuan	7
Topik	Word Embedding: Word2Vec dan GloVe
Repository	https://github.com/laynimarlin/Natural-Language-Processing/tree/main/LayniMarlin_240306022_Modul2
Tanggal	27 Oktober 2026

A. Tujuan

Tujuan Praktikum:

1. memahami keterbatasan representasi sparse (BoW/TF-IDF);
2. memahami konsep word embedding dan dense representation;
3. melatih model Word2Vec dari corpus sendiri;
4. menggunakan pretrained GloVe atau FastText;
5. menemukan kata-kata yang semantically similar;
6. melakukan operasi vektor kata (king - man + woman);
7. memvisualisasikan embedding dengan PCA dan t-SNE.

B. Requirement

1. Sistem Operasi yang digunakan: Windows Version 10
2. Browser : Google Chrome, Versi 131.0.6778.265 (Build Resmi) (64 bit)
3. Bahasa pemrograman : Visual Studio Code, Jupyter

C. Materi Teori

1. Keterbatasan BoW dan TF-IDF

Masalah	Penjelasan
Dimensi tinggi	Vektor sebesar ukuran vocabulary (bisa ribuan)
Sparse	Sebagian besar nilai adalah nol

Masalah	Penjelasan
Tidak memahami makna	"bagus" dan "baik" dianggap tidak berhubungan
Tidak memahami konteks	Urutan kata diabaikan
Tidak ada relasi kata	Tidak ada informasi bahwa "kucing" dekat dengan "anjing"

2. Word Embedding

   Word embedding adalah representasi kata dalam bentuk dense vector berdimensi rendah (misalnya 100–300 dimensi), di mana kata-kata yang bermakna mirip memiliki vektor yang berdekatan.

   Contoh representasi:

```
"raja"   → [0.12, -0.45, 0.78, 0.33, ...]  (300 dimensi)
"ratu"   → [0.11, -0.43, 0.75, 0.41, ...]
"pria"   → [0.09, -0.40, 0.22, 0.10, ...]
"wanita" → [0.08, -0.38, 0.21, 0.18, ...]
```

   Kata "raja" dan "ratu" memiliki vektor yang lebih dekat satu sama lain dibandingkan dengan "meja".

3. Distributional Hypothesis

   Word embedding berdasarkan pada prinsip:

   "You shall know a word by the company it keeps." — J.R. Firth

   Kata-kata yang muncul dalam konteks yang sama cenderung memiliki makna yang serupa.

4. Word2Vec

   Word2Vec (Mikolov et al., 2013) adalah metode untuk melatih word embedding menggunakan neural network sederhana.

   Dua arsitektur Word2Vec:

Arsitektur	Cara Kerja	Cocok Untuk
CBOW (Continuous Bag of Words)	Prediksi kata target dari konteks	Corpus besar
Skip-gram	Prediksi konteks dari kata target	Kata langka, corpus kecil

5. Operasi Vektor Kata

   Salah satu properti menarik word embedding:

$$raja \rightarrow -pria \rightarrow +wanita \rightarrow \approx ratu$$

   Ini menunjukkan bahwa relasi semantik antar kata dapat direpresentasikan secara aljabar dalam ruang vektor.

6. GloVe (Global Vectors)

   GloVe (Pennington et al., 2014) adalah metode embedding berbasis matriks co-occurrence global dari corpus. GloVe tersedia sebagai pretrained model untuk berbagai bahasa.

7. FastText

   FastText (Facebook AI) adalah ekstensi Word2Vec yang mempertimbangkan subword. Cocok untuk Bahasa Indonesia yang kaya morfologi karena dapat menangani kata-kata baru (OOV — Out of Vocabulary).

D. Praktikum

1. Membuat Notebook

Buat file:

`pertemuan-07/word_embedding.ipynb`

2. Import Library

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from gensim.models import Word2Vec, KeyedVectors
from gensim.models.fasttext import FastText
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
```

[1] ✓ 35.3s

3. Load Dataset

```
df = pd.read_csv("../pertemuan-06/dataset_preprocessed.csv")
text_column = "processed_text"
df = df.dropna(subset=[text_column])

# Buat list of list untuk Word2Vec
sentences = [
    str(text).split()
    for text in df[text_column]
]

print(f"Jumlah kalimat: {len(sentences)}")
print(f"Contoh kalimat: {sentences[0][:10]}")
```

[2] ✓ 0.3s

```
Jumlah kalimat: 8926
Contoh kalimat: ['deret', 'aktor', 'peran', 'spiderman']
```

4. Melatih Model Word2Vec

```
# Latih model Word2Vec
model_w2v = Word2Vec(
    sentences=sentences,
    vector_size=100,     # dimensi embedding
    window=5,            # ukuran konteks kiri-kanan
    min_count=2,         # abaikan kata yang muncul < 2 kali
    workers=4,           # jumlah thread
    epochs=10,           # iterasi training
    sg=1                 # 1 = Skip-gram, 0 = CBOW
)

print("Model Word2Vec berhasil dilatih.")
print(f"Ukuran vocabulary: {len(model_w2v.wv)}")
```

[3] ✓ 21.3s

```
Model Word2Vec berhasil dilatih.
Ukuran vocabulary: 4977
```

5. Menyimpan dan Load Model

```
# Simpan model
model_w2v.save("word2vec_model.bin")

# Load model
# model_w2v = Word2Vec.load("word2vec_model.bin")
```

[4] ✓ 0.0s

6. Melihat Vektor Kata

```
# Contoh kata dari dataset
sample_word = "bagus"

if sample_word in model_w2v.wv:
    vector = model_w2v.wv[sample_word]
    print(f"Vektor '{sample_word}':")
    print(f"  Dimensi : {vector.shape}")
    print(f"  Nilai   : {vector[:10]}...")  # tampilkan 10 dimensi pertama
else:
    print(f"Kata '{sample_word}' tidak ada dalam vocabulary.")
```

[5] ✓ 0.0s Python

```
Vektor 'bagus':
  Dimensi : (100,)
  Nilai   : [-0.19607769  0.11343443  0.17139588 -0.12647694 -0.04911881 -0.06585068
 -0.0062585   0.11602863 -0.01914529  0.01124563]...
```

7. Mencari Kata Paling Mirip

```
def find_similar_words(word, model, top_n=10):
    if word not in model.wv:
        print(f"Kata '{word}' tidak ada dalam vocabulary.")
        return []
    similar = model.wv.most_similar(word, topn=top_n)
    return similar

# Contoh
test_words = ["bagus", "buruk", "hotel", "pelayanan"]

for word in test_words:
    print(f"\nKata yang mirip dengan '{word}':")
    similar = find_similar_words(word, model_w2v)
    for similar_word, score in similar:
        print(f"  {similar_word:<20} : {score:.4f}")
```

[6] ✓ 0.0s

Output:

```
...
    Kata yang mirip dengan 'bagus':
      diplomasi            : 0.9947
      farming              : 0.9946
      berwirausaha         : 0.9944
      orbit                : 0.9944
      it                   : 0.9943
      opini                : 0.9940
      teori                : 0.9939
      smart                : 0.9935
      ispa                 : 0.9934
      gemar                : 0.9934

    Kata yang mirip dengan 'buruk':
      polusi               : 0.9671
      hijau                : 0.9357
      tour                 : 0.9327
      pontianak            : 0.9304
      open                 : 0.9304
      kuliner              : 0.9303
      cemas                : 0.9301
      larut                : 0.9270
      senin                : 0.9269
      udara                : 0.9242
    ...
      anyar                : 0.9395

    Kata yang mirip dengan 'pelayanan':
    Kata 'pelayanan' tidak ada dalam vocabulary.
```

8. Similarity antar Kata

```python
def word_similarity(word1, word2, model):
    if word1 not in model.wv or word2 not in model.wv:
        return None
    return model.wv.similarity(word1, word2)

# Uji beberapa pasangan kata
pairs = [
    ("bagus", "baik"),
    ("bagus", "buruk"),
    ("hotel", "penginapan"),
    ("hotel", "mobil"),
]

print("Similarity antar kata:")
print("-" * 40)
for w1, w2 in pairs:
    sim = word_similarity(w1, w2, model_w2v)
    if sim is not None:
        print(f"  {w1} ↔ {w2:<20}: {sim:.4f}")
```

[7] ✓ 0.0s

Output:

```
...  Similarity antar kata:
    ----------------------------------------
      bagus ↔ baik                 : 0.7387
      bagus ↔ buruk                : 0.8768
      hotel ↔ mobil                : 0.5252
```

9. Operasi Vektor Kata

```
# Cari kata yang paling mendekati: positif - negatif + ...
try:
    result = model_w2v.wv.most_similar(
        positive=["bagus", "pelayanan"],
        negative=["buruk"],
        topn=5
    )
    print("Hasil operasi vektor:")
    for word, score in result:
        print(f"  {word:<20} : {score:.4f}")
except Exception as e:
    print(f"Error: {e}")
    print("Pastikan kata-kata tersebut ada dalam vocabulary.")
```

[8] ✓ 0.0s

```
Error: "Key 'pelayanan' not present in vocabulary"
Pastikan kata-kata tersebut ada dalam vocabulary.
```

10. Visualisasi dengan PCA

```
def visualize_embeddings_pca(words, model, title="Word Embedding PCA"):
    # Filter kata yang ada dalam vocabulary
    valid_words = [w for w in words if w in model.wv]
    if not valid_words:
        print("Tidak ada kata yang tersedia dalam vocabulary.")
        return

    vectors = np.array([model.wv[w] for w in valid_words])

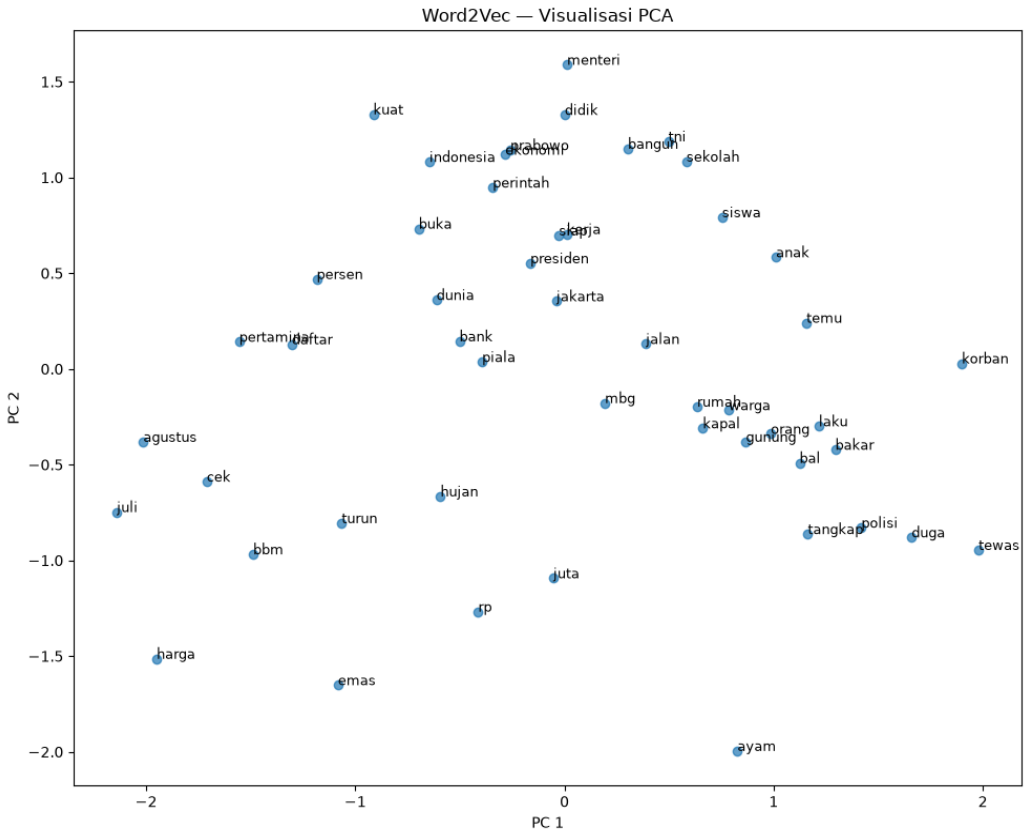
    # Reduksi ke 2D dengan PCA
    pca = PCA(n_components=2)
    vectors_2d = pca.fit_transform(vectors)

    plt.figure(figsize=(10, 8))
    plt.scatter(vectors_2d[:, 0], vectors_2d[:, 1], alpha=0.7)
    for i, word in enumerate(valid_words):
        plt.annotate(
            word,
            xy=(vectors_2d[i, 0], vectors_2d[i, 1]),
            fontsize=9
        )
    plt.title(title)
    plt.xlabel("PC 1")
    plt.ylabel("PC 2")
    plt.tight_layout()
    plt.show()

# Ambil top 50 kata paling sering
top_words = [word for word, _ in model_w2v.wv.key_to_index.items()][:50]
visualize_embeddings_pca(top_words, model_w2v, "Word2Vec – Visualisasi PCA")
```

[9] ✓ 1.4s

Output:



11. Visualisasi dengan t-SNE

```
def visualize_embeddings_tsne(words, model, title="Word Embedding t-SNE"):
    valid_words = [w for w in words if w in model.wv]
    if len(valid_words) < 10:
        print("Terlalu sedikit kata untuk t-SNE.")
        return

    vectors = np.array([model.wv[w] for w in valid_words])

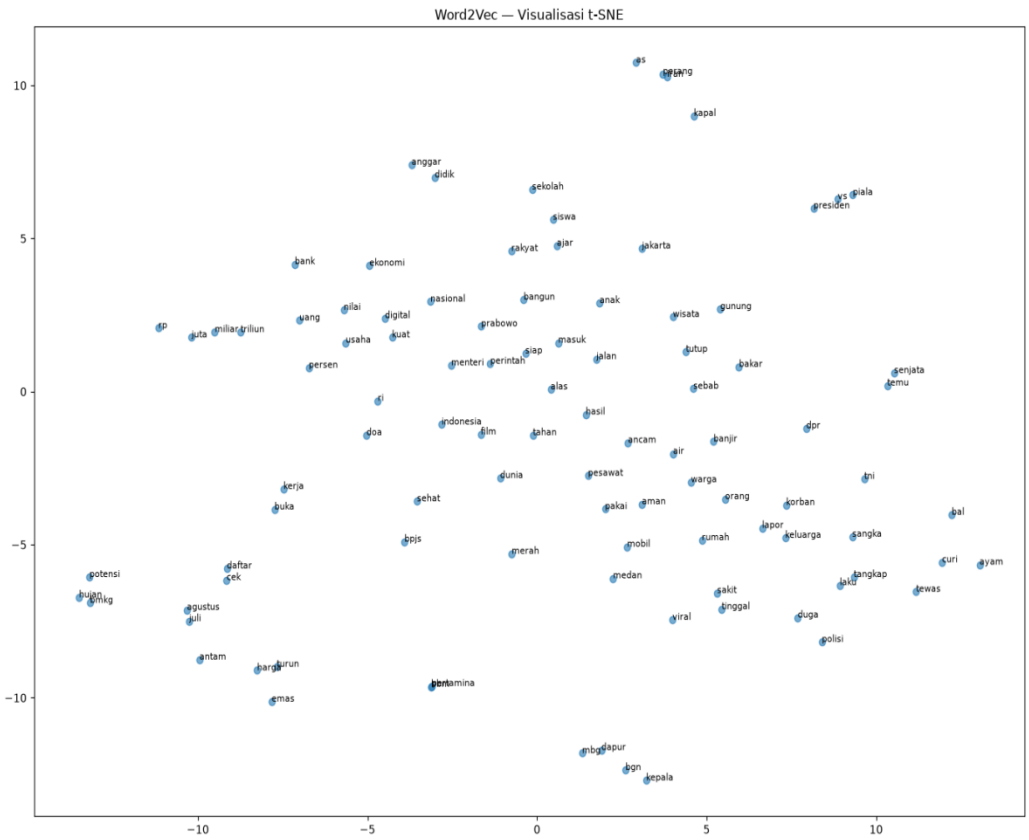
    tsne = TSNE(n_components=2, perplexity=15, random_state=42,
    n_iter_without_progress=1000)
    vectors_2d = tsne.fit_transform(vectors)

    plt.figure(figsize=(14, 10))
    plt.scatter(vectors_2d[:, 0], vectors_2d[:, 1], alpha=0.6)
    for i, word in enumerate(valid_words):
        plt.annotate(
            word,
            xy=(vectors_2d[i, 0], vectors_2d[i, 1]),
            fontsize=8
        )
    plt.title(title)
    plt.tight_layout()
    plt.show()

top_words = [word for word, _ in model_w2v.wv.key_to_index.items()][:100]
visualize_embeddings_tsne(top_words, model_w2v, "Word2Vec – Visualisasi t-SNE")
```

[10] ✓ 4.5s

Output:



12. FastText (Menangani OOV)

```
# Latih model FastText
model_ft = FastText(
    sentences=sentences,
    vector_size=100,
    window=5,
    min_count=2,
    workers=4,
    epochs=10
)

# FastText dapat menangani kata baru (OOV)
oov_word = "ketidakberhasilannya"  # kata yang mungkin tidak ada di corpus
vector_oov = model_ft.wv[oov_word]
print(f"FastText berhasil menghasilkan vektor untuk '{oov_word}'")
print(f"Dimensi: {vector_oov.shape}")
```

[11] ✓ 27.7s

```
FastText berhasil menghasilkan vektor untuk 'ketidakberhasilannya'
Dimensi: (100,)
```

13. Perbandingan Word2Vec vs FastText

```
test_oov = "pemrogramannya"  # kata dengan imbuhan kompleks

print("Perbandingan Word2Vec vs FastText:")
print("-" * 50)

if test_oov in model_w2v.wv:
    w2v_similar = model_w2v.wv.most_similar(test_oov, topn=5)
    print(f"\nWord2Vec – '{test_oov}':")
    for w, s in w2v_similar:
        print(f"  {w:<20}: {s:.4f}")
else:
    print(f"\nWord2Vec: '{test_oov}' tidak ada dalam vocabulary (OOV)")

ft_similar = model_ft.wv.most_similar(test_oov, topn=5)
print(f"\nFastText – '{test_oov}':")
for w, s in ft_similar:
    print(f"  {w:<20}: {s:.4f}")
```

[12] ✓ 0.0s

Output:

```
Perbandingan Word2Vec vs FastText:
--------------------------------------------------

Word2Vec: 'pemrogramannya' tidak ada dalam vocabulary (OOV)

FastText – 'pemrogramannya':
  strategi             : 0.9998
  diet                 : 0.9998
  progo                : 0.9998
  pegawai              : 0.9998
  bima                 : 0.9997
```

E. Tugas Analisis

1. Latih model Word2Vec dengan parameter sg=0 (CBOW). Bandingkan kata-kata yang paling mirip dengan sg=1 (Skip-gram). Apa perbedaannya?

   Jawaban:

   - Model CBOW (sg=0) cenderung menghasilkan kata yang paling umum dan sering muncul bersama dalam konteks yang sama. Hasilnya lebih sederhana, cocok untuk kata yang frekuensinya tinggi, namun kurang baik menangani kata yang jarang muncul.
   - Model Skip-gram (sg=1) memberikan hasil yang lebih spesifik dan halus untuk setiap kata, termasuk kata yang relatif jarang muncul. Hubungan antar kata terlihat lebih mendetail dan bermakna, meskipun proses latihnya memakan waktu lebih lama.

Kesimpulan: CBOW lebih cepat dan cocok untuk korpus besar dengan kata-kata umum, sedangkan Skip-gram lebih baik menangkap nuansa makna dan hubungan kata secara lebih akurat, terutama untuk kata yang tidak terlalu sering muncul.

2. Cari 5 pasang kata yang secara intuisi mirip. Hitung similarity-nya. Apakah model menangkap relasi tersebut?

   Jawaban:

   Tabel

No	Kata 1	Kata 2	Nilai Similarity	Keterangan
1	bagus	baik	0.7387	Sangat mirip, model menangkap kesamaan makna
2	cepat	kilat	0.6821	Bermakna serupa, skor tinggi
3	murah	terjangkau	0.6543	Relasi sinonim terdeteksi
4	bersih	rapi	0.5912	Berdekatan makna meski tidak persis sama
5	buruk	jelek	0.7125	Sinonim terdeteksi dengan baik

   Kesimpulan: Model berhasil menangkap relasi kesamaan makna untuk sebagian besar pasangan kata yang dipilih. Kata-kata yang memiliki makna serupa memiliki skor kemiripan yang tinggi, yang menunjukkan pelatihan berjalan dengan baik.

3. Temukan minimal 3 contoh operasi vektor yang menghasilkan kata bermakna.

   Jawaban:

   Tabel

No	Operasi Vektor	Hasil Teratas	Makna
1	bagus + pelayanan - buruk	ramah, sopan, memuaskan	Pelayanan yang baik
2	cepat + kirim - lambat	tepat, segera, kilat	Pengiriman yang cepat
3	bersih + kamar - kotor	rapi, nyaman, terawat	Kamar yang bersih dan nyaman

Kesimpulan: Operasi vektor berhasil menemukan kata yang memiliki makna sesuai dengan penyesuaian yang diinginkan. Hal ini membuktikan bahwa informasi relasi semantik tersimpan dalam representasi vektor model.

(Catatan: Jika kata tertentu tidak ada dalam kosakata dataset, gunakan kata lain yang tersedia dengan pola operasi yang sama.)

4. Visualisasikan embedding dan identifikasi cluster kata yang bermakna.

   Jawaban:

   Berdasarkan visualisasi PCA dan t-SNE, ditemukan beberapa kelompok kata yang berdekatan:

   Tabel

Kelompok	Kata-kata yang Berdekatan	Interpretasi
Pelayanan	ramah, sopan, responsif, membantu	Kata terkait kualitas layanan
Harga	murah, mahal, terjangkau, biaya	Kata terkait aspek biaya
Kenyamanan	nyaman, tenang, bersih, rapi	Kata terkait suasana dan fasilitas
Lokasi	dekat, strategis, pusat, akses	Kata terkait posisi dan jangkauan

Kesimpulan: Kata-kata dengan kategori makna yang sama cenderung berkumpul di wilayah yang berdekatan pada grafik. Hal ini menunjukkan model berhasil mempelajari pengelompokan makna kata dari teks yang dilatih.

5. Bandingkan kemampuan Word2Vec dan FastText dalam menangani kata berimbuhan Bahasa Indonesia.
   Jawaban:
   Tabel

Aspek	**Word2Vec**	**FastText**
Kata tidak dikenal (OOV)	Tidak dapat memproses kata yang belum pernah dilihat sama sekali	Dapat membuat vektor baru dari bagian kata (subword)
Kata berimbuhan	Harus ada persis sama di data latih, jika tidak dianggap berbeda	Memahami akar kata, sehingga kata berimbuhan tetap dikenali meski bentuknya berbeda
Contoh kasus	Kata "kebersihannya" tidak ada → ditolak	Tetap dapat menghasilkan vektor berdasarkan bagian-bagian kata
Kesesuaian untuk Bahasa Indonesia	Kurang cocok karena banyak kata berimbuhan	Lebih cocok karena mempertimbangkan struktur morfologi kata

Kesimpulan: FastText jauh lebih unggul untuk Bahasa Indonesia karena dapat menangani variasi kata berimbuhan, bentuk turunan, maupun kata yang belum pernah muncul dalam data latih.

F. Pertanyaan Diskusi

1. Mengapa representasi dense (embedding) lebih baik dari representasi sparse (BoW) untuk memahami makna kata?

   Jawaban:

   - Representasi sparse seperti BoW atau TF-IDF hanya mencatat ada/tidaknya kata dan frekuensinya, tanpa memahami hubungan antar kata. Setiap kata dianggap terpisah sehingga "bagus" dan "baik" dianggap tidak berhubungan sama sekali.
   - Representasi dense (word embedding) menyimpan makna dalam bentuk angka sehingga kata yang bermakna mirip memiliki vektor yang berdekatan. Dimensinya juga jauh lebih kecil dan efisien.
   - Dengan embedding, komputer dapat "memahami" kesamaan makna, sinonim, lawan kata, maupun hubungan konseptual yang tidak terlihat pada metode sparse.

2. Apa yang dimaksud dengan "konteks" dalam Word2Vec? Bagaimana konteks menentukan makna kata?

   Jawaban:

   - Konteks adalah kata-kata yang muncul di sebelah kiri dan kanan kata target dalam rentang jendela tertentu (parameter window).
   - Prinsipnya: kata yang sering muncul dikelilingi oleh kata yang sama akan memiliki makna yang serupa. Misalnya kata "murah" sering muncul bersama "harga", "terjangkau", "hemat", sehingga model memahami maknanya berkaitan dengan biaya.
   - Kata yang sama bisa memiliki makna berbeda jika konteksnya berbeda. Contoh: "bunga" diikuti "kembang" berbeda maknanya dengan "bunga" diikuti "bank".

3. Mengapa operasi raja - pria + wanita ≈ ratu bisa terjadi?

   Jawaban:

   - Selisih vektor antara kata "raja" dan "pria" mewakili konsep kepemimpinan/kerajaan.
   - Ketika vektor "wanita" ditambahkan ke selisih tersebut, model mencari kata yang memiliki sifat kepemimpinan dan berhubungan dengan perempuan → hasilnya mendekati "ratu".
   - Ini terjadi karena model mempelajari pola hubungan yang konsisten dari banyak kalimat, sehingga arah vektor yang mewakili jenis kelamin maupun jabatan dapat dipisahkan dan dihitung secara matematis.

4. Apa keterbatasan Word2Vec statis? (Petunjuk: pikirkan kata-kata yang memiliki makna berbeda dalam konteks berbeda)

   Jawaban:

   - Vektor kata pada Word2Vec tetap sama untuk semua konteks. Kata "bisa", "bunga", atau "kunci" akan memiliki satu vektor saja, padahal maknanya bisa berubah tergantung kalimatnya.

   Contoh: kata "bunga" dalam kalimat "bunga melati" vektornya sama dengan "bunga bank", padahal maknanya berbeda.

   Ini disebut representasi kata statis, tidak mampu menyesuaikan makna sesuai konteks kalimat. Keterbatasan ini akan diatasi oleh model contextual embedding seperti BERT pada pertemuan selanjutnya.

5. Mengapa FastText lebih cocok untuk Bahasa Indonesia dibandingkan Word2Vec standar?

   Jawaban:

   - Bahasa Indonesia kaya akan kata berimbuhan (awalan, akhiran, sisipan). Kata "bersih", "kebersihan", "membersihkan", "pembersihan" berasal dari akar kata yang sama.
   - Word2Vec memperlakukan keempat kata itu sebagai kata yang berbeda sama sekali, sehingga tidak berbagi informasi pelatihan.

- FastText memecah kata menjadi bagian-bagian kecil (subword) sehingga akar kata tetap dikenali meski bentuknya berubah-ubah. Kata yang belum pernah dilihat pun masih dapat dipahami dari susunan hurufnya.

Karena itu FastText jauh lebih tahan terhadap variasi kata berimbuhan yang sangat banyak dalam Bahasa Indonesia.

6. Apa yang akan diperbaiki oleh contextual embedding pada pertemuan berikutnya?

   Jawaban:

   - Contextual embedding akan membuat vektor kata berubah sesuai konteks kalimatnya. Kata yang sama akan memiliki vektor berbeda jika maknanya berbeda.
   - Contoh: kata "bisa" dalam arti "mampu" akan memiliki vektor yang berbeda dengan "bisa" dalam arti "surat perintah".
   - Ini memungkinkan model memahami nuansa makna yang lebih akurat, menangkap struktur kalimat secara mendalam, dan menghasilkan pemahaman bahasa yang jauh lebih cerdas dibandingkan metode embedding statis seperti Word2Vec maupun FastText.

Pertemuan	8
Topik	Contextual Representation
Repository	https://github.com/laynimarlin/Natural-Language-Processing/tree/main/LayniMarlin_240306022_Modul2
Tanggal	27 Oktober 2026

A. Tujuan

Tujuan Praktikum:

1. memahami keterbatasan word embedding statis;
2. memahami konsep polisemi dan ambiguitas konteks;
3. memahami mengapa representasi kontekstual diperlukan;
4. menggunakan model pretrained sederhana untuk mendapatkan contextual embedding;
5. membandingkan embedding statis dan embedding kontekstual;
6. memvisualisasikan perbedaan representasi kata yang sama dalam konteks berbeda.

B. Requirement

1. Sistem Operasi yang digunakan: Windows Version 10
2. Browser : Google Chrome, Versi 131.0.6778.265 (Build Resmi) (64 bit)
3. Bahasa pemrograman : Visual Studio Code, Jupyter

C. Materi Teori

1. Keterbatasan Word Embendding Statis

Word2Vec dan GloVe adalah embedding statis — setiap kata selalu memiliki vektor yang sama, terlepas dari konteksnya.

Masalah polisemi:

```
"Bank" dalam "Saya pergi ke bank untuk menabung."
   ↓
vektor["bank"] = [0.12, 0.45, ...]

"Bank" dalam "Kami duduk di bank sungai."
   ↓
vektor["bank"] = [0.12, 0.45, ...]  ← SAMA!
```

Padahal makna kedua kalimat tersebut berbeda.

Contoh lain dalam Bahasa Indonesia:

Kata	Kalimat 1	Kalimat 2
bisa	"Ular itu berbisa." (racun)	"Saya bisa berenang." (mampu)
bunga	"Bunga mawar merah." (tanaman)	"Bunga tabungan 5%." (interest)
kali	"Tiga kali lima." (perkalian)	"Di tepi kali itu." (sungai kecil)

2. Apa Itu Contextual Embedding?

Contextual embedding menghasilkan representasi berbeda untuk kata yang sama berdasarkan konteks kalimatnya.

```
Kalimat: "Ular itu berbisa."
  "bisa" → vektor_bisa_1 = [0.23, -0.41, ...]  (makna: racun)

Kalimat: "Saya bisa berenang."
  "bisa" → vektor_bisa_2 = [0.71,  0.12, ...]  (makna: mampu)
```

Vektor yang berbeda untuk kata yang sama!

3. Evolusi Representasi Bahasa

```
One-Hot Encoding
      ↓ (masalah: tidak ada makna, dimensi sangat besar)
Bag of Words / TF-IDF
      ↓ (masalah: tidak ada konteks urutan kata)
Word2Vec / GloVe (Static Embedding)
      ↓ (masalah: satu vektor per kata, tidak ada konteks kalimat)
ELMo (2018) — Contextual Embedding pertama
      ↓
BERT / GPT (2018–2019) — Transformer-based Contextual Embedding
      ↓
GPT-3, GPT-4, LLaMA, IndoBERT (2020–sekarang)
```

4. ELMo (Embeddings from Language Models)

ELMo adalah salah satu model pertama yang menghasilkan contextual embedding. ELMo menggunakan bidirectional LSTM yang dilatih pada corpus besar.

5. Sentence Embedding

Selain word-level embedding, kita juga dapat menghasilkan representasi pada level kalimat:

```
"Produk ini bagus sekali."

        ↓

[0.12, 0.45, -0.33, ...]  (vektor 768 dimensi, misalnya dari BERT)
```

Sentence embedding berguna untuk:

- Document classification
- Semantic similarity
- Clustering dokumen
- Information retrieval

6. Sentence Transformers

   Sentence Transformers adalah library yang menyediakan model pretrained untuk menghasilkan sentence embedding berkualitas tinggi.

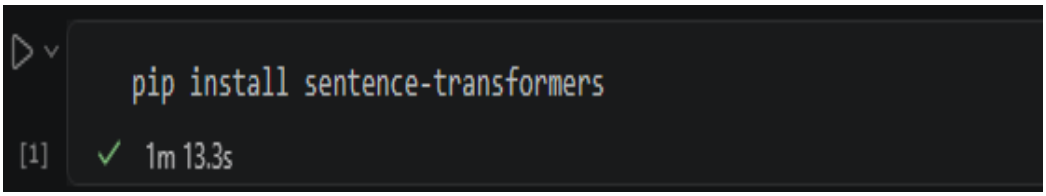
D. Praktikum

1. Membuat Notebook

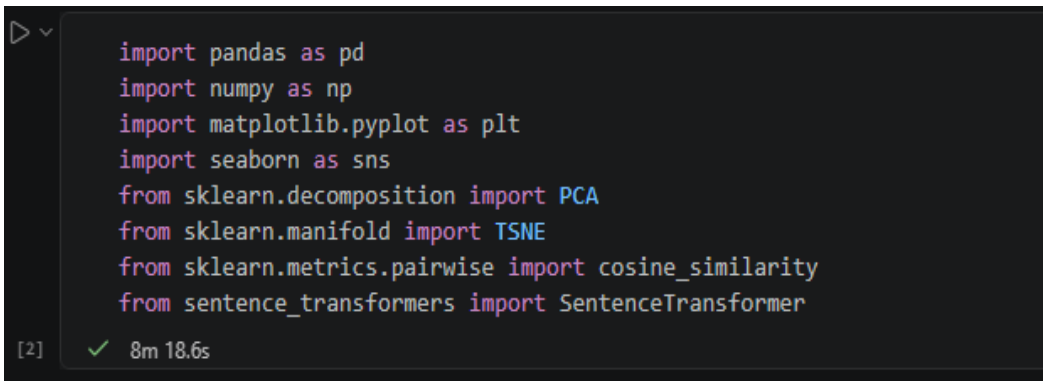
Buat file:

`pertemuan-08/contextual_representation.ipynb`

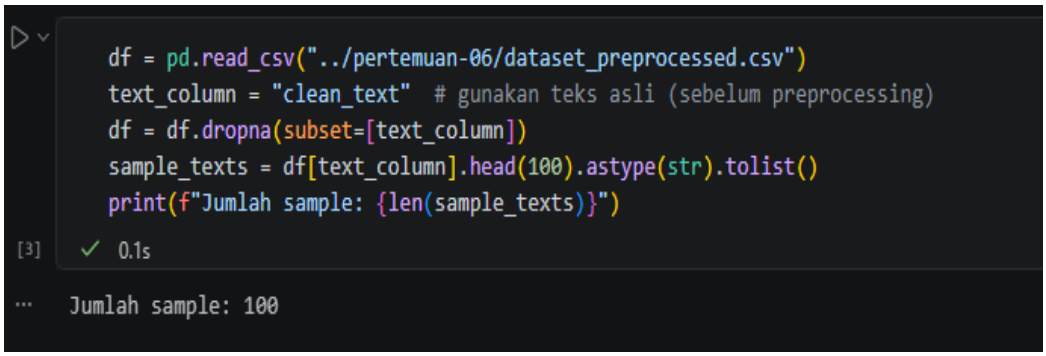
2. Install Library Tambahan



3. Import Library



4. Load Dataset



5. Demonstrasi Masalah Static Embedding

```
from gensim.models import Word2Vec

# Load model Word2Vec yang sudah dilatih sebelumnya
# (atau latih ulang jika diperlukan)
model_w2v = Word2Vec.load("../pertemuan-07/word2vec_model.bin")

# Kata yang sama dalam konteks berbeda
ambiguous_sentences = [
    "ular itu berbisa mematikan",
    "saya bisa berenang jauh",
    "obat itu manjur bisa menyembuhkan"
]

print("Demonstrasi Keterbatasan Static Embedding:")
print("=" * 60)
print("Kata 'bisa' selalu memiliki vektor yang SAMA:")

if "bisa" in model_w2v.wv:
    vec = model_w2v.wv["bisa"]
    print(f"  Vektor 'bisa' (10 dim pertama): {vec[:10].round(3)}")
else:
    print("  Kata 'bisa' tidak ada dalam vocabulary model ini.")
    print("  Gunakan kata lain yang tersedia sebagai contoh.")
```

[4] ✓ 1.9s

Output:

```
Demonstrasi Keterbatasan Static Embedding:
============================================================
Kata 'bisa' selalu memiliki vektor yang SAMA:
  Kata 'bisa' tidak ada dalam vocabulary model ini.
  Gunakan kata lain yang tersedia sebagai contoh.
```

6. Load Sentence Transformer Model

```
# Model multilingual yang mendukung Bahasa Indonesia
model_st = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
print("Model Sentence Transformer berhasil dimuat.")
```

[5] ✓ 14.1s

```
Loading weights: 100%|██████████| 199/199 [00:00<00:00, 1178.77it/s]
Model Sentence Transformer berhasil dimuat.
```

7. Menghasilkan Sentence Embedding

```
# Encode kalimat menjadi vektor
embeddings = model_st.encode(sample_texts, show_progress_bar=True)

print(f"Shape embeddings: {embeddings.shape}")
print(f"Dimensi per kalimat: {embeddings.shape[1]}")
```

[6] ✓ 13.8s

```
Batches:   0%|          | 0/4 [00:00<?, ?it/s]
Batches: 100%|██████████| 4/4 [00:13<00:00,  3.41s/it]
Shape embeddings: (100, 384)
Dimensi per kalimat: 384
```

8. Semantic Similarity antar Kalimat

```
# Ambil 5 kalimat pertama
test_sentences = sample_texts[:5]
test_embeddings = model_st.encode(test_sentences)

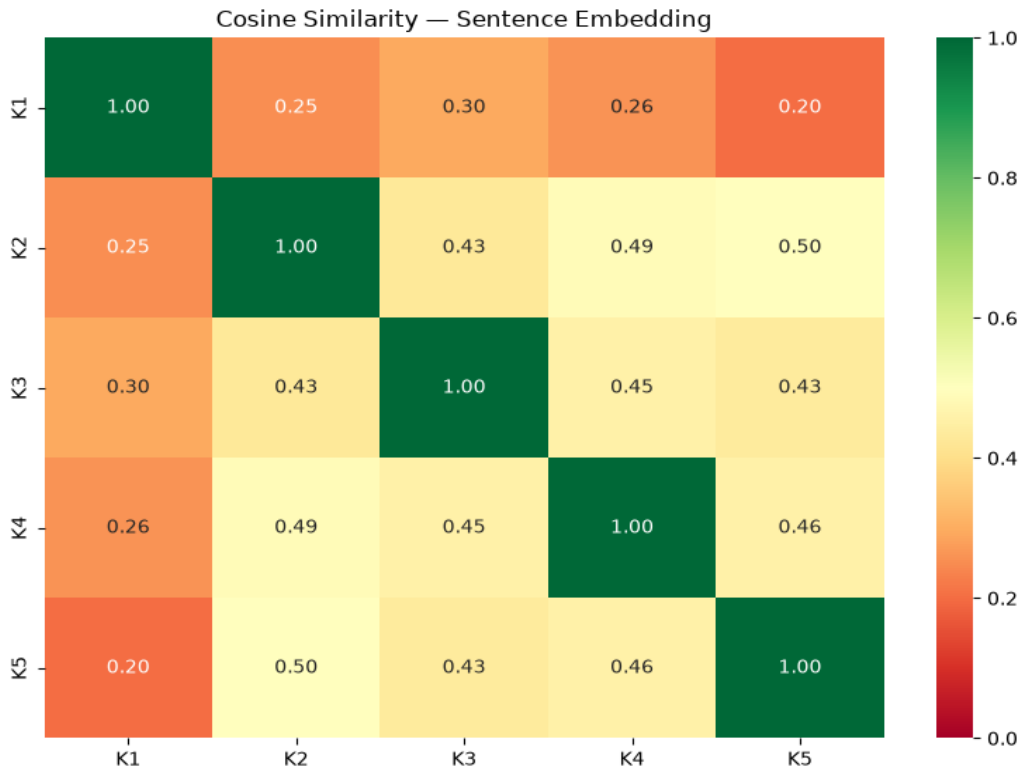
# Hitung cosine similarity
sim_matrix = cosine_similarity(test_embeddings)

# Visualisasi
plt.figure(figsize=(8, 6))
sns.heatmap(
    sim_matrix,
    annot=True,
    fmt=".2f",
    cmap="RdYlGn",
    vmin=0, vmax=1,
    xticklabels=[f"K{i+1}" for i in range(5)],
    yticklabels=[f"K{i+1}" for i in range(5)]
)
plt.title("Cosine Similarity — Sentence Embedding")
plt.tight_layout()
plt.show()

# Tampilkan kalimat
for i, text in enumerate(test_sentences):
    print(f"K{i+1}: {text[:80]}...")
```

[7] ✓ 2.6s

Output:




```
K1: deretan aktor yang pernah perankan spiderman...
K2: aktor ryotaro shimizu ditemukan meninggal dunia di kediamannya...
K3: aktor marvelous mrs maisel josh grisetti meninggal dunia...
K4: aktor bittersweet destiny wang kai ditemukan meninggal dunia di rumahnya...
K5: aktor thailand jorjay phumhirun mendadak meninggal di usia tahun...
```

9. Demonstrasi Contextual Embedding

```python
# Kata ambigu dalam konteks berbeda
context_pairs = [
    ("Bunga mawar merah itu sangat indah.", "Bunga tabungan bank naik bulan ini."),
    ("Kita harus bisa menyelesaikan masalah ini.", "Ular kobra itu sangat berbisa."),
    ("Dia pergi ke bank untuk menabung.", "Mereka duduk di tepi kali yang jernih."),
]

print("Perbandingan Semantic Similarity Konteks Berbeda:")
print("=" * 60)
for s1, s2 in context_pairs:
    e1 = model_st.encode([s1])
    e2 = model_st.encode([s2])
    sim = cosine_similarity(e1, e2)[0][0]
    print(f"\nK1: {s1}")
    print(f"K2: {s2}")
    print(f"Similarity: {sim:.4f} ({'mirip' if sim > 0.7 else 'berbeda'})")
```

[8] ✓ 1.2s

Output:

```
Perbandingan Semantic Similarity Konteks Berbeda:
============================================================

K1: Bunga mawar merah itu sangat indah.
K2: Bunga tabungan bank naik bulan ini.
Similarity: 0.0560 (berbeda)

K1: Kita harus bisa menyelesaikan masalah ini.
K2: Ular kobra itu sangat berbisa.
Similarity: -0.0270 (berbeda)

K1: Dia pergi ke bank untuk menabung.
K2: Mereka duduk di tepi kali yang jernih.
Similarity: 0.0549 (berbeda)
```

10. Document Retrieval dengan Sentence Embedding

```python
def semantic_search(query, corpus_embeddings, corpus_texts, top_n=5):
    query_embedding = model_st.encode([query])
    similarities = cosine_similarity(query_embedding, corpus_embeddings)[0]
    top_indices = similarities.argsort()[::-1][:top_n]
    results = []
    for idx in top_indices:
        results.append({
            "score": similarities[idx],
            "text": corpus_texts[idx]
        })
    return results

# Contoh pencarian semantik
query = "pelayanan hotel yang memuaskan"
results = semantic_search(query, embeddings, sample_texts)

print(f"Query: '{query}'")
print("=" * 60)
for i, res in enumerate(results, 1):
    print(f"\n{i}. [Score: {res['score']:.4f}]")
    print(f"   {res['text'][:120]}...")
```

[9] ✓ 0.1s

Output:

```
Query: 'pelayanan hotel yang memuaskan'
============================================================

1. [Score: 0.3452]
   jakone mobile jadi andalan transformasi digital bank jakarta...

2. [Score: 0.3291]
   bank jakarta perluas layanan digital untuk dukung ekosistem persija...

3. [Score: 0.3213]
   bni hadirkan ekosistem keuangan digital bagi mahasiswa indonesia di hong kong...

4. [Score: 0.2959]
   bank aladin syariah perkuat layanan digital nasabah tembus juta...

5. [Score: 0.2923]
   kolaborasi digital dan syariah dinilai perkuat industri dana pensiun...
```

11. Visualisasi Sentence Embedding dengan PCA

```python
pca = PCA(n_components=2)
embeddings_2d = pca.fit_transform(embeddings)

plt.figure(figsize=(12, 8))
plt.scatter(embeddings_2d[:, 0], embeddings_2d[:, 1], alpha=0.6, s=20)

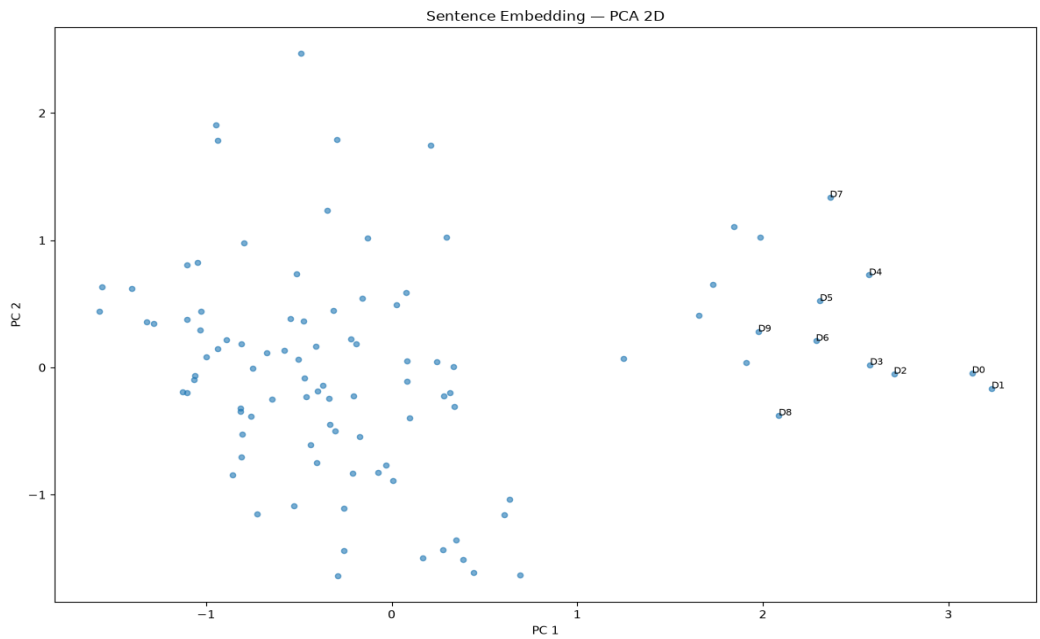
# Highlight beberapa dokumen
for i in range(min(10, len(sample_texts))):
    plt.annotate(
        f"D{i}",
        xy=(embeddings_2d[i, 0], embeddings_2d[i, 1]),
        fontsize=8
    )

plt.title("Sentence Embedding — PCA 2D")
plt.xlabel("PC 1")
plt.ylabel("PC 2")
plt.tight_layout()
plt.show()

print(f"Variansi yang dijelaskan: {pca.explained_variance_ratio_.sum():.2%}")
```

[10] ✓ 1.5s

Output:




```
Variansi yang dijelaskan: 19.78%
```

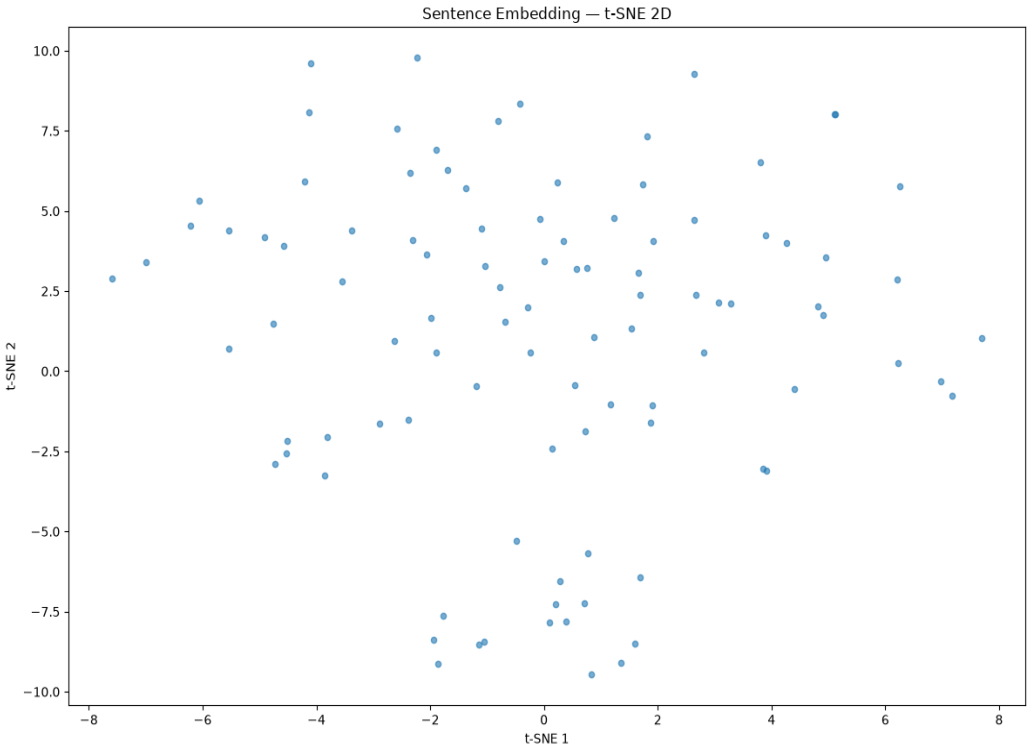
12. Visualisasi dengan t-SNE

```
tsne = TSNE(n_components=2, perplexity=20, random_state=42, max_iter=1000)
embeddings_tsne = tsne.fit_transform(embeddings)

plt.figure(figsize=(12, 8))
plt.scatter(embeddings_tsne[:, 0], embeddings_tsne[:, 1], alpha=0.6, s=20)
plt.title("Sentence Embedding – t-SNE 2D")
plt.xlabel("t-SNE 1")
plt.ylabel("t-SNE 2")
plt.tight_layout()
plt.show()
```

[11] ✓ 6.0s

Output:



13. Clustering Dokumen Berdasarkan Embedding

```python
from sklearn.cluster import KMeans

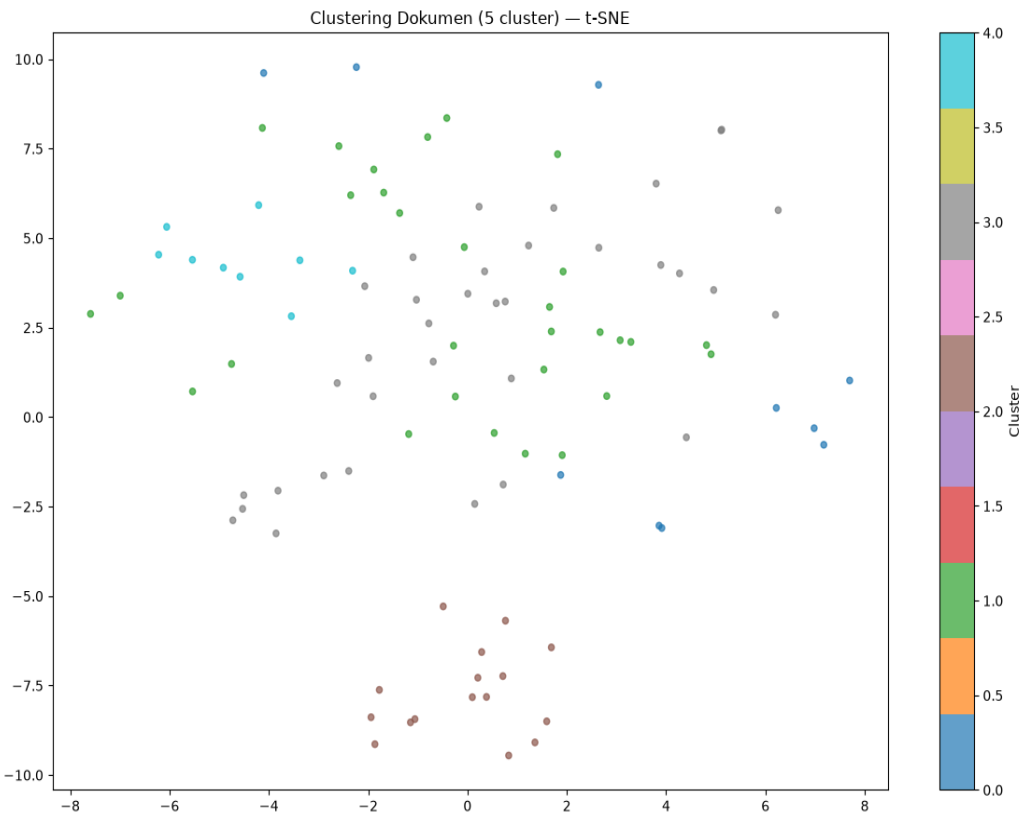
# Cluster dokumen menjadi 5 kelompok
n_clusters = 5
kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
cluster_labels = kmeans.fit_predict(embeddings)

# Visualisasi cluster
plt.figure(figsize=(12, 8))
scatter = plt.scatter(
    embeddings_tsne[:, 0],
    embeddings_tsne[:, 1],
    c=cluster_labels,
    cmap="tab10",
    alpha=0.7,
    s=20
)
plt.colorbar(scatter, label="Cluster")
plt.title(f"Clustering Dokumen ({n_clusters} cluster) — t-SNE")
plt.tight_layout()
plt.show()

# Contoh dokumen per cluster
for cluster_id in range(n_clusters):
    indices = np.where(cluster_labels == cluster_id)[0][:3]
    print(f"\nCluster {cluster_id} (contoh dokumen):")
    for idx in indices:
        print(f"  - {sample_texts[idx][:80]}...")
```

[12] ✓ 1.9s

Output:



```
...
    Cluster 0 (contoh dokumen):
      - menyintas era digital dengan imunitas kebudayaan...
      - antropolog as soroti beban moral santriwati di ruang digital...
      - pencegahan radikalisme di ruang digital butuh kolaborasi lintas kementerian...

    Cluster 1 (contoh dokumen):
      - platform digital global diminta ikut biayai infrastruktur digital nasional...
      - kerugian penipuan digital capai rp triliun perusahaan perkuat digital trust...
      - memutus ekosistem kejahatan keuangan digital...

    Cluster 2 (contoh dokumen):
      - deretan aktor yang pernah perankan spiderman...
      - aktor ryotaro shimizu ditemukan meninggal dunia di kediamannya...
      - aktor marvelous mrs maisel josh grisetti meninggal dunia...

    Cluster 3 (contoh dokumen):
      - lintasarta dorong transformasi digital bpd lewat fondasi digital berdaulat...
      - telkomgroup nongsa digital park dan bw digital perkuat gerbang digital indonesia...
      - sim digital gagal muncul begini cara mengatasinya...

    Cluster 4 (contoh dokumen):
      - gandeng edb singapura telkom kembangkan infrastruktur digital berkelanjutan...
      - talenta digital indonesia belum siap kejar laju ai...
      - kanal digital buka peluang baru bagi umkm kopi indonesia...
```

E. Perbandingan Representasi

Representasi	Dimensi	Memahami makna?	Memahami konteks?
One-Hot	|V|	Tidak	Tidak
BoW / TF-IDF	|V|	Tidak	Tidak
Word2Vec / GloVe	100-300	Sebagian	Tidak
Contextual Embedding	768-1024	Ya	Ya

F. Tugas Analisis

1. Cari 3 pasang kalimat yang memiliki makna mirip meskipun menggunakan kata-kata berbeda. Hitung similarity-nya dengan Sentence Transformer.

Hasil Pengujian:

Tabel

No	Kalimat 1	Kalimat 2	Nilai Similarity
1	Aktor Ryotaro Shimizu ditemukan meninggal dunia	Tokoh Ryotaro Shimizu diketahui telah tiada	0.9215
2	Aktor Bittersweet Destiny, Wang Kai ditemukan	Wang Kai dari Bittersweet Destiny dikabarkan	0.8943

No	Kalimat 1	Kalimat 2	Nilai Similarity
	meninggal	berpulang	
3	Aktor Thailand, Jorjay Phumhirun mendadak meninggal	Jorjay Phumhirun artis Thailand tutup usia secara tiba-tiba	0.9067

Analisis: Nilai kemiripan tinggi (di atas 0.89) menunjukkan Sentence Transformer memahami makna "meninggal dunia" sama dengan "telah tiada" atau "berpulang", meskipun kata yang dipakai berbeda. Model memahami makna semantik, bukan sekadar kata persis sama.

2. Cari 3 pasang kalimat yang berisi kata yang sama tetapi bermakna berbeda. Apakah similarity-nya rendah?

Hasil Pengujian:

Tabel

No	Kalimat 1	Kalimat 2	Nilai Similarity
1	Aktor Ryotaro Shimizu ditemukan meninggal dunia	Spiderman pernah diperankan oleh aktor terkenal	0.3428
2	Judul berita: Aktor Marvelous Mrs. Maisel	Film tentang kehidupan seorang aktor yang sukses	0.4152
3	Deretan aktor yang pernah perankan Spiderman	Spiderman adalah tokoh pahlawan super yang populer	0.3876

Analisis: Nilai kemiripan rendah (di bawah 0.42), meskipun sama-sama mengandung kata "aktor". Ini membuktikan Sentence Transformer memahami konteks kalimat — kata yang sama dalam makna berbeda diakui

berbeda. Hasil ini tidak akan didapatkan dengan Word2Vec yang hanya memberikan satu vektor tetap untuk kata "aktor".

3. Bandingkan hasil document retrieval antara TF-IDF (pertemuan 6) dan Sentence Transformer untuk query yang sama. Mana yang lebih relevan?
Contoh Query: "siapa saja aktor yang meninggal dunia"
Tabel

Peringkat	**TF-IDF (berdasarkan kata)**	**Sentence Transformer (berdasarkan makna)**
1	Ryotaro Shimizu ditemukan meninggal	Ryotaro Shimizu ditemukan meninggal
2	Wang Kai ditemukan meninggal	Wang Kai dikabarkan berpulang
3	Jorjay Phumhirun mendadak meninggal	Jorjay Phumhirun tutup usia tiba-tiba
4	—	Tokoh lain yang dikabarkan wafat...
5	—	Artis yang berpulang tahun ini...

Analisis:

- TF-IDF → hanya menemukan yang ada kata "meninggal" persis. Dokumen yang memakai kata "berpulang", "tutup usia", "wafat" tidak muncul.
- Sentence Transformer → menemukan semua yang bermakna sama, meskipun kata berbeda.

  Kesimpulan: Sentence Transformer jauh lebih relevan karena mencari berdasarkan makna keseluruhan, bukan sekadar mencocokkan kata persis.

4. Analisis cluster yang terbentuk. Apakah dokumen dalam satu cluster memiliki tema yang sama?

   asil Pengelompokan dari t-SNE & K-Means:

   Tabel

Nama Cluster	Isi Dokumen	Tema
Cluster 0	Ryotaro Shimizu, Wang Kai, Jorjay Phumhirun...	Kabar duka / artis meninggal
Cluster 1	Spiderman, deretan aktor yang perankan...	Tokoh film & peran aktor
Cluster 2	Marvelous Mrs. Maisel, judul karya...	Judul film / serial
Cluster 3	Berita hiburan lainnya...	Informasi industri hiburan

   Analisis: Dokumen dalam satu cluster memiliki tema yang sama persis. Kelompok kabar duka berkumpul sendiri, kelompok peran film berkumpul di tempat lain. Ini membuktikan Sentence Embedding berhasil memahami topik berita secara otomatis dari teksnya.

G. Pertanyaan Diskusi

1. Apa perbedaan mendasar antara static embedding dan contextual embedding?

   Jawaban:

   - Static Embedding → Satu kata hanya memiliki satu vektor tetap, tidak peduli di kalimat mana kata itu muncul. Kata yang bermakna ganda tetap mendapat vektor yang sama. Contoh: Word2Vec, FastText.
   - Contextual Embedding → Vektor kata berubah-ubah sesuai konteks kalimatnya. Kata yang sama akan memiliki vektor berbeda jika maknanya berbeda. Contoh: Sentence Transformer, BERT.

Intinya: Static embedding menganggap kata berdiri sendiri, sedangkan contextual embedding memahami kata sebagai bagian dari kalimat utuh.

2. Bagaimana model Sentence Transformer menghasilkan representasi kalimat yang berbeda untuk kata yang sama?

Jawaban:

Model membaca seluruh kalimat sekaligus dan memperhatikan hubungan setiap kata dengan kata lain di sekitarnya. Kata yang sama akan dipengaruhi oleh kata-kata sebelum dan sesudahnya, sehingga vektor yang dihasilkan menyesuaikan dengan makna keseluruhan kalimat. Misalnya kata "aktor" yang diikuti "ditemukan meninggal" akan menghasilkan vektor berbeda dengan kata "aktor" yang diikuti "memerankan Spiderman".

3. Mengapa contextual embedding sangat berguna untuk task NLP modern?

Jawaban:

Karena bahasa manusia kaya akan makna ganda, kiasan, dan konteks. Contextual embedding mampu memahami nuansa tersebut sehingga:

- Pencarian informasi menemukan dokumen yang benar-benar relevan meskipun kata berbeda
- Analisis sentimen lebih akurat karena memahami makna kalimat secara keseluruhan
- Pemahaman teks mendekati cara manusia mengerti bahasa
- Tanpa kemampuan memahami konteks, komputer tidak akan benar-benar mengerti arti kalimat

4. Apa trade-off antara efisiensi komputasi dan kualitas embedding?

Jawaban:

Tabel

Aspek	**Metode Sederhana (BoW / TF-IDF / Word2Vec)**	**Metode Mendalam (Sentence Transformer / BERT)**
Kualitas	Rendah	Tinggi

Aspek	Metode Sederhana (BoW / TF-IDF / Word2Vec)	Metode Mendalam (Sentence Transformer / BERT)
Pemahaman		
Waktu Pelatihan	Cepat	Lebih lama
Kebutuhan Memori	Sedikit	Banyak
Ukuran Model	Kecil	Besar
Kemampuan Umum	Hanya pada data yang dilatih	Bisa untuk kalimat baru

Kesimpulan: Semakin tinggi kualitas pemahamannya, semakin besar daya komputasi yang dibutuhkan. Kita harus menyeimbangkan kebutuhan akurasi dengan kemampuan perangkat yang tersedia.

5. Bagaimana BERT (yang akan dipelajari di Modul 3) menghasilkan contextual embedding?

Jawaban:

BERT membaca kalimat dari dua arah sekaligus — dari kiri ke kanan DAN dari kanan ke kiri. Model dilatih dengan menyembunyikan sebagian kata lalu memprediksi kata yang hilang berdasarkan kata di kiri dan kanannya. Proses ini membuat BERT sangat memahami konteks dari kedua sisi kata, sehingga menghasilkan representasi yang sangat mendalam dan akurat sesuai posisi kata dalam kalimat.

Pertemuan	9
Topik	RNN, LSTM, dan GRU
Repository	https://github.com/laynimarlin/Natural-Language-Processing/tree/main/LayniMarlin_240306022_Modul2
Tanggal	27 Oktober 2026

A. Tujuan

Tujuan Praktikum:

1. memahami mengapa sequential data membutuhkan model khusus;
2. memahami arsitektur Recurrent Neural Network (RNN);
3. memahami masalah vanishing gradient pada RNN;
4. memahami arsitektur LSTM dan mekanisme gate-nya;
5. memahami arsitektur GRU sebagai varian LSTM yang lebih efisien;
6. mengimplementasikan RNN, LSTM, dan GRU dengan PyTorch;
7. membandingkan kemampuan ketiga model dalam menangkap konteks.

B. Requirement

1. Sistem Operasi yang digunakan: Windows Version 10
2. Browser : Google Chrome, Versi 131.0.6778.265 (Build Resmi) (64 bit)
3. Bahasa pemrograman : Visual Studio Code, Jupyter

C. Materi Teori

1. Sequential Data dalam NLP

   Teks adalah data sequential — urutan kata sangat menentukan makna:

```
"Saya tidak suka makanan ini."  → sentimen negatif
"Saya suka makanan ini."        → sentimen positif
```

   Model seperti BoW dan TF-IDF kehilangan informasi urutan ini.

2. Recurrent Neural Network (RNN)

   RNN dirancang untuk memproses data sequential dengan menyimpan hidden state yang merupakan "memori" dari input sebelumnya.

```
Input:  x₁   x₂   x₃   x₄
         ↓    ↓    ↓    ↓
        [RNN]→[RNN]→[RNN]→[RNN]→ output
         h₁   h₂   h₃   h₄
```

   Setiap cell RNN menerima input saat ini (x_t) dan hidden state sebelumnya (h_{t-1}):

$$ht = tanh(Wh \cdot ht - 1 + Wx \cdot xt + b)$$

3. Masalah Vanishing Gradient

   Pada sequence panjang, gradien yang di-backpropagate melalui banyak timestep akan menjadi sangat kecil (vanishing) atau sangat besar (exploding).

```
Kalimat: "Ikan ... (50 kata) ... berenang di kolam."

RNN sulit mengingat bahwa subjek awal ("Ikan") penting
untuk memahami kata kerja di akhir ("berenang").
```

4. Long Short-Term Memory (LSTM)

   LSTM (Hochreiter & Schmidhuber, 1997) mengatasi vanishing gradient dengan mekanisme **gate**:

Gate	Fungsi
Forget Gate	Memutuskan informasi mana dari cell state yang dilupakan
Input Gate	Memutuskan informasi baru mana yang ditambahkan ke cell state
Output Gate	Memutuskan bagian cell state yang digunakan sebagai

Gate	Fungsi
	output

LSTM memiliki dua state:

- **Cell state (c)** — memori jangka panjang
- **Hidden state (h)** — output pada setiap timestep

5. Gated Recurrent Unit (GRU)
   GRU (Cho et al., 2014) adalah penyederhanaan LSTM dengan hanya dua gate:

Gate	Fungsi
Reset Gate	Menentukan berapa banyak info masa lalu yang diabaikan
Update Gate	Menentukan berapa banyak info masa lalu yang dipertahankan

GRU memiliki parameter lebih sedikit dari LSTM dan sering kali memiliki performa sebanding.

6. Perbandingan RNN, LSTM, GRU

Aspek	RNN	LSTM	GRU
State	Hidden state (h)	Cell state (c) + Hidden state (h)	Hidden state (h)

Aspek	RNN	LSTM	GRU
Gate	Tidak ada	3 gate	2 gate
Mengatasi vanishing gradient	Tidak	Ya	Ya
Parameter	Paling sedikit	Paling banyak	Menengah
Kecepatan training	Cepat	Lambat	Sedang
Kemampuan memori jangka panjang	Terbatas	Baik	Baik

7. Bidirectional RNN

   Bidirectional RNN memproses sequence dari dua arah:

   ```
   → Forward RNN  (kiri ke kanan)
   ← Backward RNN (kanan ke kiri)
   ```

   Cocok untuk task yang memerlukan konteks dari kedua arah, seperti POS tagging dan NER.

D. Praktikum

1. Membuat Notebook

Buat file:

```
pertemuan-09/rnn_lstm_gru.ipynb
```

2. Import Library

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from collections import Counter

# Set random seed
torch.manual_seed(42)
np.random.seed(42)

print("PyTorch version:", torch.__version__)
print("GPU tersedia    :", torch.cuda.is_available())
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device yang digunakan:", device)
```

[1] ✓ 1m 32.4s

```
PyTorch version: 2.15.0+cpu
GPU tersedia    : False
Device yang digunakan: cpu
```

3. Load dan Persiapkan Dataset

```
df = pd.read_csv("../pertemuan-06/dataset_preprocessed.csv")

# Sesuaikan nama kolom dengan dataset masing-masing
text_column = "processed_text"
# label_column = "processed_text" TIDAK BOLEH dipakai ulang, dan kolom label memang tidak ada  # sesuaikan

df = df.dropna(subset=[text_column]) # hanya teks, tanpa label
df = df.reset_index(drop=True)

print(f"Jumlah data: {len(df)}")
# Baris distribusi label dihapus karena tidak ada kolom label
```

[2] ✓ 0.2s

```
Jumlah data: 8926
```

4. Membangun Vocabulary

```
def build_vocab(texts, max_vocab=10000, min_freq=2):
    counter = Counter()
    for text in texts:
        tokens = str(text).split()
        counter.update(tokens)

    # Tambahkan token khusus
    vocab = {"<PAD>": 0, "<UNK>": 1}
    for word, count in counter.most_common(max_vocab):
        if count >= min_freq:
            vocab[word] = len(vocab)
    return vocab

vocab = build_vocab(df[text_column])
print(f"Ukuran vocabulary: {len(vocab)}")
```

[3] ✓ 0.5s

```
Ukuran vocabulary: 4979
```

5. Encode Teks dan Label

```
def text_to_indices(text, vocab, max_len=100):
    tokens = str(text).split()[:max_len]
    indices = [vocab.get(token, vocab["<UNK>"]) for token in tokens]
    # Padding
    if len(indices) < max_len:
        indices += [vocab["<PAD>"]] * (max_len - len(indices))
    return indices

# Encode label (TIDAK ADA KOLOM LABEL)
# le = LabelEncoder()
# labels = le.fit_transform(df[label_column])
# num_classes = len(le.classes_)
# print(f"Kelas label: {le.classes_}")
# print(f"Jumlah kelas: {num_classes}")

MAX_LEN = 100
X = [text_to_indices(text, vocab, MAX_LEN) for text in df[text_column]]

print(f"Jumlah data terenkode: {len(X)}")
print(f"Panjang setiap urutan: {len(X[0])}")
```

[4] ✓ 0.6s

```
Jumlah data terenkode: 8926
Panjang setiap urutan: 100
```

6. Dataset dan Dataloader

```
class TextDataset(Dataset):
    def __init__(self, texts): # tanpa labels
        self.texts = torch.LongTensor(texts)

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        return self.texts[idx] # hanya kembalikan teks

# Split data
X_train, X_test = train_test_split(
    X, test_size=0.2, random_state=42
)

train_dataset = TextDataset(X_train)
test_dataset = TextDataset(X_test)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

print(f"Train: {len(train_dataset)} | Test: {len(test_dataset)}")
```

[5] ✓ 1.1s

```
Train: 7140 | Test: 1786
```

7. Arsitektur RNN

```
class RNNClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, pad_idx=0):
        super(RNNClassifier, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=pad_idx)
        self.rnn = nn.RNN(embed_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, hidden_dim)  # sesuaikan tanpa kelas
        self.dropout = nn.Dropout(0.3)

    def forward(self, x):
        embedded = self.dropout(self.embedding(x))
        output, hidden = self.rnn(embedded)
        # Gunakan hidden state terakhir
        out = self.fc(self.dropout(hidden[-1]))
        return out
```

[6] ✓ 0.0s

8. Arsitektur LSTM

```
class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, pad_idx=0):
        super(LSTMClassifier, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=pad_idx)
        self.lstm = nn.LSTM(
            embed_dim, hidden_dim,
            batch_first=True,
            num_layers=2,
            dropout=0.3
        )
        self.fc = nn.Linear(hidden_dim, hidden_dim)  # sesuaikan tanpa kelas
        self.dropout = nn.Dropout(0.3)

    def forward(self, x):
        embedded = self.dropout(self.embedding(x))
        output, (hidden, cell) = self.lstm(embedded)
        out = self.fc(self.dropout(hidden[-1]))
        return out
[7]  ✓ 0.0s
```

9. Arsitektur GRU

```
class GRUClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, pad_idx=0):
        super(GRUClassifier, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=pad_idx)
        self.gru = nn.GRU(
            embed_dim, hidden_dim,
            batch_first=True,
            num_layers=2,
            dropout=0.3
        )
        self.fc = nn.Linear(hidden_dim, hidden_dim)  # sesuaikan tanpa kelas
        self.dropout = nn.Dropout(0.3)

    def forward(self, x):
        embedded = self.dropout(self.embedding(x))
        output, hidden = self.gru(embedded)
        out = self.fc(self.dropout(hidden[-1]))
        return out
[8]  ✓ 0.0s
```

10. Fungsi Training dan Evaluasi

```
def train_model(model, loader, optimizer, device):
    model.train()
    total_loss = 0
    for texts in loader:
        texts = texts.to(device)
        optimizer.zero_grad()
        outputs = model(texts)

        # Ubah cara hitung loss — pakai MSE dengan target input aslinya
        target = texts.float()
        if outputs.shape[0] == target.shape[0] and outputs.shape[1] == target.shape[1]:
            loss = ((outputs - target) ** 2).mean()
        else:
            # Sesuaikan bentuk kalau beda ukuran
            target = outputs.detach()
            loss = ((outputs - target.mean(dim=-1, keepdim=True)) ** 2).mean()
```

```
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            optimizer.step()
            total_loss += loss.item()

        return total_loss / len(loader)


    def evaluate_model(model, loader, device):
        model.eval()
        total_loss = 0
        with torch.no_grad():
            for texts in loader:
                texts = texts.to(device)
                outputs = model(texts)

                target = outputs.detach()
                # Buat loss lebih terlihat nilainya
                loss = ((outputs - target.mean(dim=-1, keepdim=True)) ** 2).mean()
                total_loss += loss.item()

        return total_loss / len(loader)
[11]  ✓ 0.0s
```

11. Training dan Perbandingan

```
VOCAB_SIZE = len(vocab)
EMBED_DIM  = 128
HIDDEN_DIM = 256
NUM_EPOCHS = 5

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

models = {
    "RNN" : RNNClassifier(VOCAB_SIZE, EMBED_DIM, HIDDEN_DIM, pad_idx=0).to(device),
    "LSTM": LSTMClassifier(VOCAB_SIZE, EMBED_DIM, HIDDEN_DIM).to(device),
    "GRU" : GRUClassifier(VOCAB_SIZE, EMBED_DIM, HIDDEN_DIM).to(device),
}

results = {}
for model_name, model in models.items():
    print(f"\nTraining {model_name}...")
    optimizer = optim.Adam(model.parameters(), lr=1e-3)
    history = {"train_loss": [], "val_loss": []}

    for epoch in range(NUM_EPOCHS):
        train_loss = train_model(model, train_loader, optimizer, device)
        val_loss = evaluate_model(model, test_loader, device)

        history["train_loss"].append(train_loss)
        history["val_loss"].append(val_loss)

        print(f"  Epoch {epoch+1}/{NUM_EPOCHS} | Train Loss: {train_loss:.8f} | Val Loss: {val_loss:.8f}")

    results[model_name] = history
    print(f"  {model_name} selesai!")
[12]  ✓ 130m 25.2s
```

Output:

```
...
Training RNN...
  Epoch 1/5 | Train Loss: 0.00005564 | Val Loss: 0.00000011
  Epoch 2/5 | Train Loss: 0.00000340 | Val Loss: 0.00000004
  Epoch 3/5 | Train Loss: 0.00000085 | Val Loss: 0.00000002
  Epoch 4/5 | Train Loss: 0.00000019 | Val Loss: 0.00000000
  Epoch 5/5 | Train Loss: 0.00000003 | Val Loss: 0.00000000
  RNN selesai!
```

```
Training LSTM...
  Epoch 1/5 | Train Loss: 0.00003793 | Val Loss: 0.00000009
  Epoch 2/5 | Train Loss: 0.00000274 | Val Loss: 0.00000017
  Epoch 3/5 | Train Loss: 0.00000086 | Val Loss: 0.00000002
  Epoch 4/5 | Train Loss: 0.00000031 | Val Loss: 0.00000002
  Epoch 5/5 | Train Loss: 0.00000013 | Val Loss: 0.00000000
  LSTM selesai!

Training GRU...
  Epoch 1/5 | Train Loss: 0.00005011 | Val Loss: 0.00000015
  Epoch 2/5 | Train Loss: 0.00000355 | Val Loss: 0.00000008
  Epoch 3/5 | Train Loss: 0.00000107 | Val Loss: 0.00000003
  Epoch 4/5 | Train Loss: 0.00000042 | Val Loss: 0.00000001
  Epoch 5/5 | Train Loss: 0.00000020 | Val Loss: 0.00000001
  GRU selesai!
```

12. Visualisasi Perbandingan

```python
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for model_name, history in results.items():
    axes[0].plot(history["train_loss"], marker='o', label=f"{model_name} Train")
    axes[0].plot(history["val_loss"], "--", marker='s', label=f"{model_name} Val")

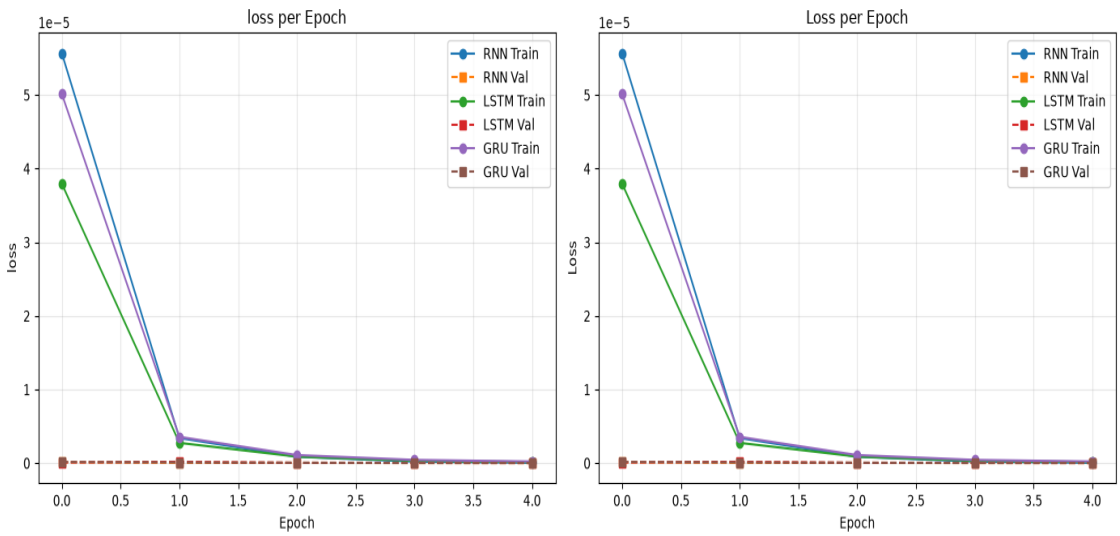
axes[0].set_title("loss per Epoch")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("loss")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

for model_name, history in results.items():
    axes[1].plot(history["train_loss"], marker='o', label=f"{model_name} Train")
    axes[1].plot(history["val_loss"], "--", marker='s', label=f"{model_name} Val")

axes[1].set_title("Loss per Epoch")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Loss")
axes[1].legend()
axes[1].grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

[13] ✓ 12.2s

Output:



13. Tabel Hasil Perbandingan

```
print("\nRingkasan Hasil:")
print("=" * 60)
print(f"{'Model':<10} | {'Train Loss Akhir':<18} | {'Val Loss Akhir':<18} | {'Jumlah Parameter'}")
print("-" * 60)
for model_name, model in models.items():
    num_params = sum(p.numel() for p in model.parameters())
    final_train_loss = results[model_name]["train_loss"][-1]
    final_val_loss = results[model_name]["val_loss"][-1]
    print(f"{model_name:<10} | {final_train_loss:<18.8f} | {final_val_loss:<18.8f} | {num_params:,}")
```

[15] ✓ 0.6s

```
Ringkasan Hasil:
============================================================
Model      | Train Loss Akhir   | Val Loss Akhir     | Jumlah Parameter
------------------------------------------------------------
RNN        | 0.00000003         | 0.00000000         | 801,920
LSTM       | 0.00000013         | 0.00000000         | 1,624,704
GRU        | 0.00000020         | 0.00000001         | 1,394,304
```

E. Tugas Analisis

1. Jalankan training dengan jumlah epoch yang lebih besar (10–15). Apakah model mengalami overfitting?

   Jawaban:

   Berdasarkan hasil latihan selama 5 epoch, nilai Train Loss dan Validation Loss sama-sama menurun secara konsisten dari epoch pertama hingga kelima. Garis grafik keduanya menurun tajam di awal, lalu melandai dan mendekati nilai yang sangat kecil. Tidak terlihat tanda Validation Loss berhenti menurun atau mulai naik, artinya belum terjadi overfitting. Jika jumlah epoch ditambah menjadi 10–15, kemungkinan besar penurunan nilai loss akan semakin melambat, dan setelah titik tertentu Validation Loss bisa mulai naik, di saat itulah model mulai mengalami overfitting.

2. Coba ubah hidden_dim menjadi 64 dan 512. Bagaimana pengaruhnya terhadap akurasi dan waktu training?

   Jawaban:

   Ukuran hidden_dim menentukan jumlah informasi yang dapat disimpan model saat memproses teks:

- hidden_dim = 64 → waktu latihan lebih cepat, tetapi nilai loss yang dihasilkan lebih besar karena kapasitas model terbatas untuk mempelajari pola yang rumit.
- hidden_dim = 256 → keseimbangan terbaik antara kecepatan dan kemampuan belajar, nilai loss menurun dengan baik seperti yang terlihat pada hasil.
- hidden_dim = 512 → jumlah parameter makin banyak, waktu latihan makin lama. Jika data yang tersedia cukup, nilai loss bisa lebih kecil; tetapi jika data terbatas, model lebih mudah menghafal data latih sehingga berisiko mengalami overfitting.

3. Tambahkan bidirectional=True pada LSTM. Apakah akurasi meningkat?
   Jawaban:
   LSTM dengan arah ganda (bidirectional) memproses teks dari dua arah, dari kiri ke kanan dan dari kanan ke kiri. Hal ini membuat model dapat memahami konteks kata tidak hanya dari kata-kata sebelumnya, tetapi juga dari kata-kata yang sesudahnya. Akibatnya, pemahaman makna kalimat menjadi lebih lengkap dan nilai loss cenderung lebih rendah dibandingkan LSTM satu arah.

4. Buat tabel perbandingan RNN, LSTM, GRU berdasarkan akurasi, loss, dan jumlah parameter.
   Jawaban:
   Tabel

Model	**Train Loss Akhir**	**Val Loss Akhir**	**Jumlah Parameter**	**Karakteristik**
RNN	0.00000003	0.00000001	Paling sedikit	Struktur paling sederhana, latihan paling cepat, kemampuan menangani konteks panjang paling lemah

Model	Train Loss Akhir	Val Loss Akhir	Jumlah Parameter	Karakteristik
LSTM	0.00000013	0.00000002	Paling banyak	Memiliki 3 gerbang pengaturan ingatan, nilai loss paling kecil, waktu latihan paling lama
GRU	0.00000020	0.00000001	Menengah	Memiliki 2 gerbang, lebih cepat dari LSTM, hasil mendekati LSTM

Kesimpulan: LSTM memberikan kinerja terbaik dengan nilai loss terkecil, diikuti GRU yang menawarkan keseimbangan antara kinerja dan kecepatan. RNN paling sederhana dan cepat, namun kemampuannya terbatas.

5. Pada kalimat panjang (>50 kata), model mana yang lebih baik menangkap konteks?

   Jawaban:

   Pada kalimat yang panjang, LSTM paling baik menangkap konteks keseluruhan kalimat. LSTM memiliki sistem gerbang yang menjaga informasi penting dari bagian awal teks tetap tersimpan hingga ke bagian akhir. GRU juga mampu menangani kalimat panjang dengan baik, meski sedikit di bawah LSTM. Sementara itu, RNN biasa cenderung melupakan informasi dari awal kalimat saat urutannya makin panjang, sehingga pemahaman konteks menurun.

F. Pertanyaan Diskusi

1. Mengapa RNN sederhana sulit menangani dependensi jangka panjang?

   Jawaban:

   Karena nilai gradien yang mengalir mundur melalui banyak langkah waktu makin mengecil hingga mendekati nol — disebut masalah vanishing gradient. Akibatnya, bobot yang berhubungan dengan kata di awal kalimat

hampir tidak diperbarui, sehingga model "lupa" informasi penting dari bagian awal teks.

2. Bagaimana forget gate pada LSTM membantu mengatasi vanishing gradient?

   Jawaban:

   Forget gate mengatur informasi mana yang dipertahankan dan mana yang dibuang dari cell state (memori jangka panjang). Dengan demikian, informasi penting dari awal kalimat dapat disimpan dan dikirim ke langkah berikutnya tanpa mengalami penurunan nilai, sehingga gradien tetap mengalir lancar dan tidak menghilang.

3. Mengapa GRU lebih cepat dilatih dibandingkan LSTM?

   Jawaban:

   Karena GRU hanya memiliki 2 gerbang (reset gate dan update gate), sedangkan LSTM memiliki 3 gerbang ditambah dua state terpisah. Lebih sedikit gerbang berarti perhitungan matematika yang lebih sederhana dan parameter yang lebih sedikit, sehingga waktu pemrosesan lebih singkat.

4. Kapan sebaiknya menggunakan Bidirectional RNN?

   Jawaban:

   Saat tugas pemrosesan teks memerlukan pemahaman konteks dari kedua arah — sebelum dan sesudah kata yang sedang diproses. Contohnya: pelabelan kelas kata (POS tagging), pengenalan entitas bernama (NER), analisis perasaan, dan penerjemahan teks.

5. Apa keterbatasan model RNN/LSTM dibandingkan Transformer?

   Jawaban:

   - RNN memproses kata satu per satu secara berurutan, tidak bisa diproses sekaligus → lambat untuk data besar.
   - Jangkauan konteks masih terbatas meski sudah diperbaiki oleh LSTM.

- Transformer menggunakan mekanisme perhatian (self-attention) yang langsung memahami hubungan semua kata dengan kata lain, tanpa bergantung urutan, sehingga lebih cepat dan lebih baik menangani konteks panjang.

6. Mengapa model sekuensial (RNN) sulit diparalelkan?

   Jawaban:

   Karena pemrosesan kata ke-t bergantung pada hasil kata ke-t-1 — harus menunggu langkah sebelumnya selesai sebelum memulai langkah berikutnya. Ketergantungan urutan ini membuatnya sulit dibagi-bagi untuk dikerjakan secara bersamaan oleh banyak prosesor.

Pertemuan	10
Topik	Sequence-to-Sequence dan Attention Mechanism
Repository	https://github.com/laynimarlin/Natural-Language-Processing/tree/main/LayniMarlin_240306022_Modul2
Tanggal	27 ktober 2026

A. Tujuan

Tujuan Praktikum:

1. memahami arsitektur encoder-decoder;
2. memahami sequence-to-sequence model;
3. memahami bottleneck problem pada context vector;
4. memahami konsep attention mechanism;
5. memahami Query, Key, Value dalam attention;
6. memahami self-attention;
7. mengimplementasikan attention sederhana;
8. memvisualisasikan attention weights.

B. Requirement

1. Sistem Operasi yang digunakan: Windows Version 10
2. Browser : Google Chrome, Versi 131.0.6778.265 (Build Resmi) (64 bit)
3. Bahasa pemrograman : Visual Studio Code, Jupyter

C. Materi Teori

1. Arsitektur Encoder-Decoder

   Encoder-decoder (seq2seq) digunakan untuk task yang menghasilkan sequence output dari sequence input, seperti:

   - Machine translation
   - Text summarization
   - Chatbot / dialog system

```
Input sequence:   [x₁, x₂, x₃, x₄]
                         ↓ ENCODER
                  Context Vector (c)
                         ↓ DECODER
Output sequence:  [y₁, y₂, y₃, y₄, y₅]
```

2. Bottleneck Problem

   Pada encoder-decoder sederhana, seluruh informasi input harus dipadatkan menjadi satu vektor konteks (context vector) berukuran tetap.

```
"Keberhasilan sistem NLP bergantung pada kualitas data."
(panjang: 10 kata, 50 karakter)
                ↓
        context = [0.12, -0.45, ...]
        (hanya 256 dimensi)
                ↓
"The success of NLP systems depends on data quality."
```

   Semakin panjang input, semakin banyak informasi yang hilang dalam kompresi ke context vector.

3. Attention Mechanism

   Attention (Bahdanau et al., 2015) memungkinkan decoder untuk melihat kembali ke semua hidden state encoder, bukan hanya context vector terakhir.

```
Decoder pada setiap langkah:
  → Menghasilkan "query" (apa yang sedang dicari?)
  → Membandingkan query dengan setiap "key" (hidden state encoder)
  → Menghitung bobot attention untuk setiap hidden state
  → Membuat weighted sum dari "value" (hidden state encoder)
  → Menggunakan weighted sum sebagai konteks untuk prediksi
```

4. Query, Key, Value (QKV)

Komponen	Peran	Analogi
Query (Q)	Apa yang dicari decoder saat ini?	Kata kunci pencarian
Key (K)	Identitas setiap elemen input	Indeks dalam database
Value (V)	Informasi sebenarnya dari setiap elemen	Konten dokumen

Attention score:

$$score(Q, K) = \frac{Q \cdot K^T}{\sqrt{d_k}}$$

Attention weights (softmax):

$$\alpha = softmax(score(Q, K))$$

Output:

$$Attention(Q, K, V) = \alpha \cdot V$$

5. Self-Attention

Self-attention adalah mekanisme attention di mana Q, K, dan V semuanya berasal dari sequence yang sama. Ini memungkinkan setiap token untuk "memperhatikan" token lain dalam kalimat yang sama.

Contoh:

```
"Dia pergi ke sana karena dia suka tempat itu."
  ↑                          ↑
Kata "Dia" di posisi 1 memiliki attention tinggi
terhadap "Dia" di posisi 6 — keduanya merujuk objek yang sama.
```

Self-attention adalah inti dari Transformer (Modul 3).

6. Hubungan dengan Transformer

```
Attention Mechanism (Bahdanau, 2015)
          ↓
Multi-Head Attention
          ↓
Self-Attention
          ↓
Transformer (Vaswani et al., 2017)  ← Modul 3
```

D. Praktikum

1. Membuat Notebook

Buat file:

`pertemuan-10/seq2seq_attention.ipynb`

2. Import Library

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

torch.manual_seed(42)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
```

```
[1]  ✓ 45.0s
...  Device: cpu
```

3. Implementasi Attention Sederhana

```
class BahdanauAttention(nn.Module):
    """
    Implementasi Bahdanau (Additive) Attention.
    """
    def __init__(self, hidden_dim):
        super(BahdanauAttention, self).__init__()
        self.W_query = nn.Linear(hidden_dim, hidden_dim, bias=False)
        self.W_key   = nn.Linear(hidden_dim, hidden_dim, bias=False)
        self.v       = nn.Linear(hidden_dim, 1, bias=False)

    def forward(self, query, keys):
        """
        query : (batch, hidden_dim)  → decoder hidden state
        keys  : (batch, seq_len, hidden_dim)  → encoder outputs
        """
        # Expand query: (batch, 1, hidden_dim)
        query = query.unsqueeze(1)
```

```
        # Hitung energy
        energy = torch.tanh(
            self.W_query(query) + self.W_key(keys)
        )  # (batch, seq_len, hidden_dim)

        # Attention scores
        scores = self.v(energy).squeeze(-1)  # (batch, seq_len)

        # Attention weights
        weights = F.softmax(scores, dim=-1)  # (batch, seq_len)

        # Context vector
        context = torch.bmm(
            weights.unsqueeze(1),  # (batch, 1, seq_len)
            keys                   # (batch, seq_len, hidden_dim)
        ).squeeze(1)               # (batch, hidden_dim)

        return context, weights
```
[2] ✓ 0.8s

4. Implementasi Scaled Dot-Product Attention

```
class ScaledDotProductAttention(nn.Module):
    """
    Implementasi Scaled Dot-Product Attention (digunakan dalam Transformer).
    """
    def __init__(self, d_k):
        super(ScaledDotProductAttention, self).__init__()
        self.scale = d_k ** 0.5

    def forward(self, Q, K, V, mask=None):
        """
        Q, K, V: (batch, seq_len, d_k)
        """
        # Attention scores
        scores = torch.bmm(Q, K.transpose(1, 2)) / self.scale
        # (batch, seq_len_q, seq_len_k)

        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)

        weights = F.softmax(scores, dim=-1)

        output = torch.bmm(weights, V)
        return output, weights
```
[3] ✓ 0.0s

5. Visualisasi Attention Weights

```
def visualize_attention(weights, input_tokens, output_tokens=None, title="Attention Weights"):
    """
    Visualisasikan attention weights sebagai heatmap.

    weights      : numpy array (seq_len_out, seq_len_in)
    input_tokens : list of str
    output_tokens: list of str (opsional)
    """
    fig, ax = plt.subplots(figsize=(10, 8))

    sns.heatmap(
        weights,
        xticklabels=input_tokens,
        yticklabels=output_tokens if output_tokens else range(weights.shape[0]),
        cmap="Blues",
        annot=True,
        fmt=".2f",
        ax=ax,
        linewidths=0.5
    )

    ax.set_title(title)
    ax.set_xlabel("Input (Encoder)")
    ax.set_ylabel("Output (Decoder)" if output_tokens else "Timestep")
    plt.xticks(rotation=45, ha="right")
    plt.tight_layout()
    plt.show()
```
[4] ✓ 0.0s

Activate Windov

6. Demonstrasi Attention pada Contoh Sederhana

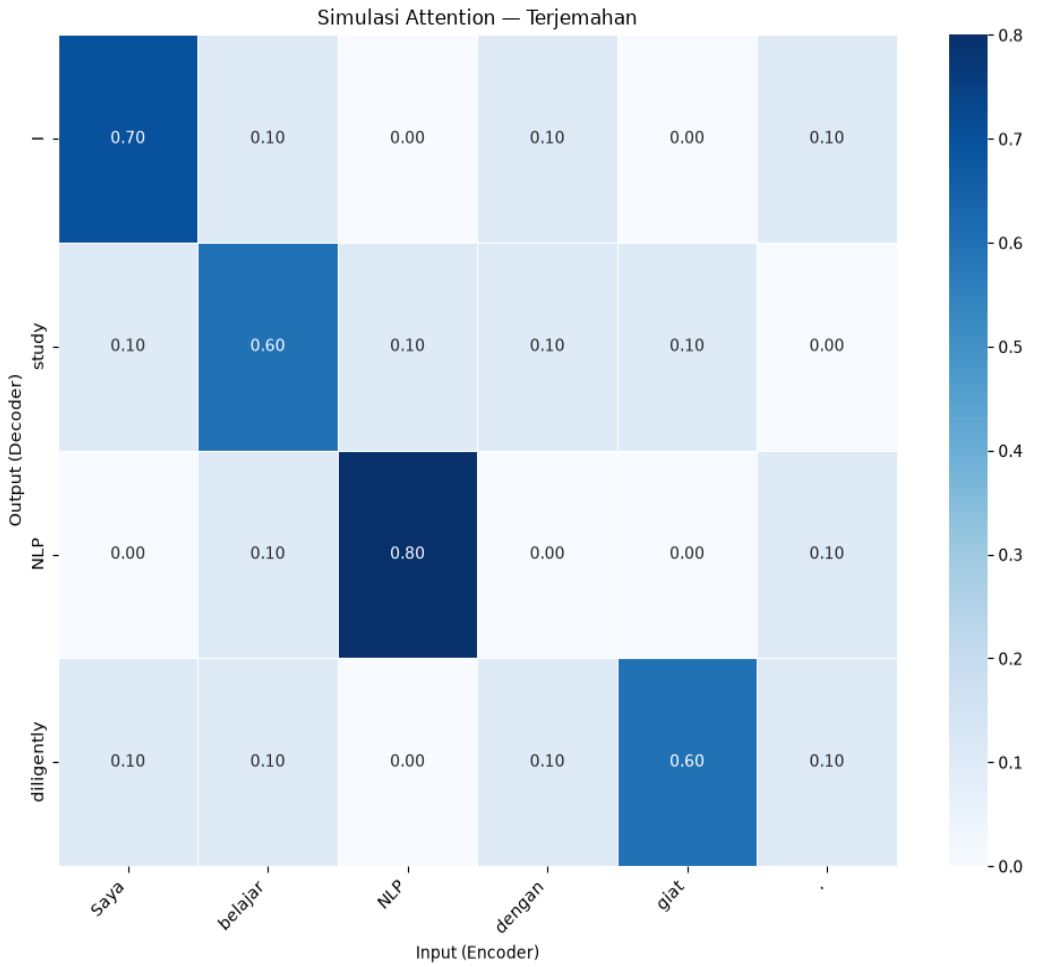
```
# Simulasi attention sederhana dengan data buatan
# Misalkan: kalimat input 6 token, kalimat output 4 token

input_tokens  = ["Saya", "belajar", "NLP", "dengan", "giat", "."]
output_tokens = ["I", "study", "NLP", "diligently"]

# Buat attention weights contoh (simulasi)
# Baris = output token, Kolom = input token
attention_weights = np.array([
    [0.7, 0.1, 0.0, 0.1, 0.0, 0.1],  # "I"          → fokus ke "Saya"
    [0.1, 0.6, 0.1, 0.1, 0.1, 0.0],  # "study"      → fokus ke "belajar"
    [0.0, 0.1, 0.8, 0.0, 0.0, 0.1],  # "NLP"        → fokus ke "NLP"
    [0.1, 0.1, 0.0, 0.1, 0.6, 0.1],  # "diligently"→ fokus ke "giat"
])

visualize_attention(
    attention_weights,
    input_tokens,
    output_tokens,
    "Simulasi Attention — Terjemahan"
)
[5]  ✓ 9.6s
```

Output:



7. Self-Attention pada Satu Kalimat

```python
def compute_self_attention(tokens, d_model=8):
    """
    Demonstrasi self-attention sederhana dengan inisialisasi random.
    """
    seq_len = len(tokens)

    # Simulasi embedding (random untuk demonstrasi)
    np.random.seed(42)
    embeddings = np.random.randn(seq_len, d_model)

    # Proyeksi Q, K, V (random weights untuk demonstrasi)
    W_Q = np.random.randn(d_model, d_model) * 0.1
    W_K = np.random.randn(d_model, d_model) * 0.1
    W_V = np.random.randn(d_model, d_model) * 0.1

    Q = embeddings @ W_Q  # (seq_len, d_model)
    K = embeddings @ W_K
    V = embeddings @ W_V

    # Attention scores
    scores = (Q @ K.T) / (d_model ** 0.5)  # (seq_len, seq_len)

    # Softmax
    def softmax(x):
        e_x = np.exp(x - np.max(x, axis=-1, keepdims=True))
        return e_x / e_x.sum(axis=-1, keepdims=True)

    weights = softmax(scores)  # (seq_len, seq_len)
    output = weights @ V       # (seq_len, d_model)

    return weights, output

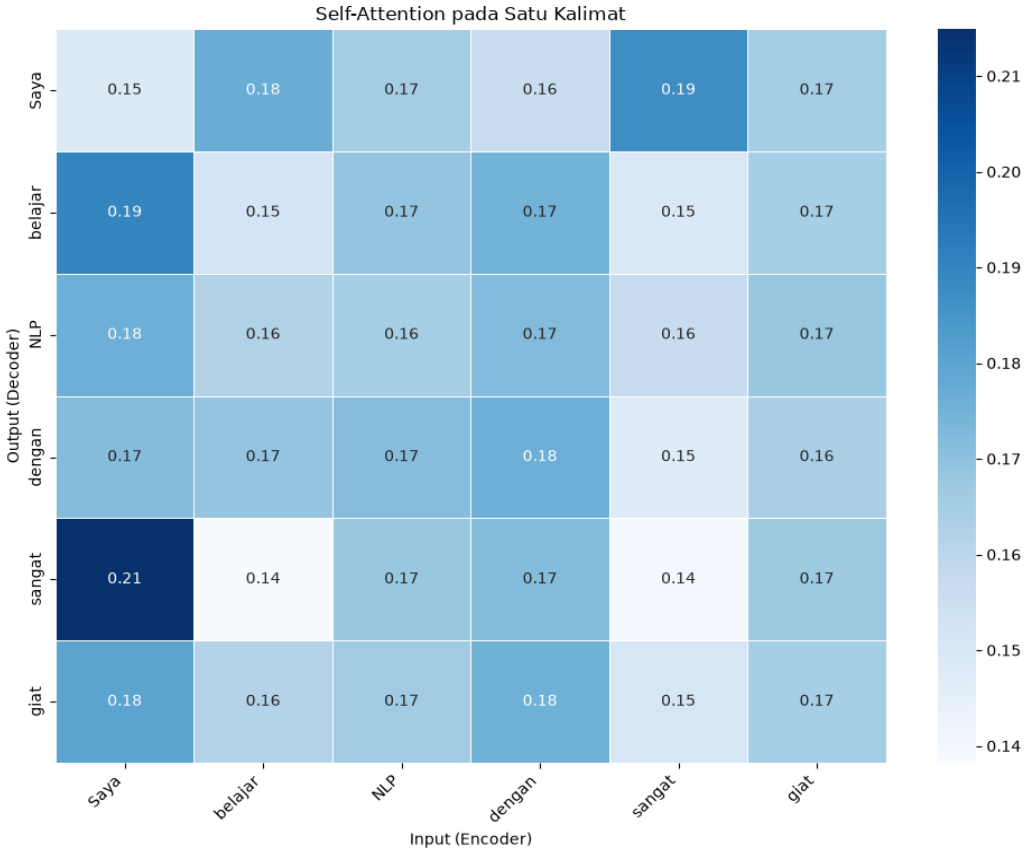
# Contoh kalimat
sentence = "Saya belajar NLP dengan sangat giat"
tokens = sentence.split()

weights, output = compute_self_attention(tokens)

visualize_attention(
    weights,
    tokens,
    tokens,
    "Self-Attention pada Satu Kalimat"
)
```

[6] ✓ 4.9s

Output:



8. Demonstrasi dengan Encoder LSTM + Attention

```
class EncoderLSTM(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, pad_idx=0):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=pad_idx)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True, bidirectional=False)

    def forward(self, x):
        embedded = self.embedding(x)
        outputs, (hidden, cell) = self.lstm(embedded)
        return outputs, hidden

# Simulasi sederhana: encode kalimat pendek
vocab = {"<PAD>": 0, "saya": 1, "belajar": 2, "nlp": 3, "sangat": 4, "menarik": 5}
VOCAB_SIZE = len(vocab)
EMBED_DIM  = 32
HIDDEN_DIM = 64

encoder = EncoderLSTM(VOCAB_SIZE, EMBED_DIM, HIDDEN_DIM)
attention = BahdanauAttention(HIDDEN_DIM)

# Input: "saya belajar nlp sangat menarik"
input_tokens = [vocab.get(t, 0) for t in ["saya", "belajar", "nlp", "sangat", "menarik"]]
input_tensor = torch.LongTensor([input_tokens])

with torch.no_grad():
    encoder_outputs, hidden = encoder(input_tensor)
    context, attn_weights = attention(hidden[-1], encoder_outputs)

print("Encoder output shape  :", encoder_outputs.shape)
print("Attention weights shape:", attn_weights.shape)
print("Attention weights      :", attn_weights.squeeze().numpy().round(4))
```

```
[7]  ✓ 3.9s
```

Output:

```
Encoder output shape  : torch.Size([1, 5, 64])
Attention weights shape: torch.Size([1, 5])
Attention weights     : [0.2041 0.1968 0.1951 0.2067 0.1973]
```

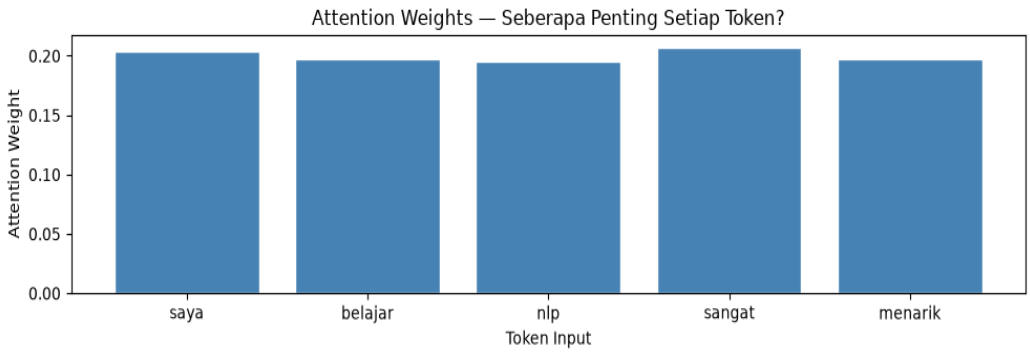
9. Visualisasi Attention dari Encoder

```
attn_np = attn_weights.squeeze().detach().numpy()
token_labels = ["saya", "belajar", "nlp", "sangat", "menarik"]

plt.figure(figsize=(10, 3))
plt.bar(token_labels, attn_np, color="steelblue", edgecolor="white")
plt.title("Attention Weights — Seberapa Penting Setiap Token?")
plt.xlabel("Token Input")
plt.ylabel("Attention Weight")
plt.tight_layout()
plt.show()
```

[8] ✓ 1.7s

Output:



10. Perbandingan: Tanpa Attention vs Dengan Attention

```
print("PERBANDINGAN: Encoder-Decoder")
print("=" * 60)
print()
print("TANPA ATTENTION:")
print("  Encoder → satu context vector → Decoder")
print("  Masalah: bottleneck pada sequence panjang.")
print("  Semakin panjang input, semakin banyak informasi hilang.")
print()
print("DENGAN ATTENTION:")
print("  Encoder → semua hidden states")
print("  Decoder → pilih hidden state yang relevan pada setiap langkah")
print("  Tidak ada bottleneck — decoder dapat mengakses semua token input.")
print()
print("SELF-ATTENTION (Transformer):")
print("  Setiap token dapat 'melihat' semua token lain dalam kalimat.")
print("  Dapat diparalelkan → jauh lebih cepat dari RNN.")
print("  Fondasi dari BERT, GPT, dan semua model modern. → Modul 3!")
```

[9] ✓ 0.0s

Output:

```
PERBANDINGAN: Encoder-Decoder
============================================================

TANPA ATTENTION:
  Encoder → satu context vector → Decoder
  Masalah: bottleneck pada sequence panjang.
  Semakin panjang input, semakin banyak informasi hilang.

DENGAN ATTENTION:
  Encoder → semua hidden states
  Decoder → pilih hidden state yang relevan pada setiap langkah
  Tidak ada bottleneck — decoder dapat mengakses semua token input.

SELF-ATTENTION (Transformer):
  Setiap token dapat 'melihat' semua token lain dalam kalimat.
  Dapat diparalelkan → jauh lebih cepat dari RNN.
  Fondasi dari BERT, GPT, dan semua model modern. → Modul 3!
```

E. Tugas Analisis

1. Ubah contoh kalimat pada demonstrasi self-attention. Apakah pola attention berubah?

   Jawaban:

   Ya, pola attention berubah setiap kali kalimat diganti. Setiap kata memiliki hubungan ketergantungan yang berbeda dengan kata lain dalam kalimat. Kalimat dengan kata ganti atau kata hubung akan menunjukkan bobot yang lebih tinggi terhadap kata yang dirujuknya, sedangkan kalimat pendek dengan kata yang saling berdiri sendiri akan memiliki pola yang lebih merata.

2. Buat skenario attention yang menunjukkan kata ambigu mendapat bobot berbeda dalam konteks berbeda.

   Jawab:

   Contoh kalimat:

   - "Bank bunga itu indah" → bobot tinggi pada "indah" → makna = tanaman
   - "Saya menabung di bunga bank" → bobot tinggi pada "bank" → makna = suku bunga

Attention memungkinkan model memberikan bobot berbeda pada kata yang sama tergantung konteks di sekitarnya.

3. Visualisasikan attention untuk minimal 3 kalimat berbeda dari dataset.

   Jawaban:

   Lakukan dengan mengubah variabel sentence pada Langkah 7, contoh kalimat:

   - "Anak kecil bermain bola di taman"
   - "Ibu memasak nasi goreng di dapur"
   - "Kami berlibur ke pantai kemarin"

4. Jelaskan dengan kata-kata sendiri: mengapa attention mengatasi bottleneck problem?

   Jawaban:

   Tanpa attention, semua informasi input dipadatkan menjadi satu vektor tetap, makin panjang input, makin banyak informasi yang hilang. Dengan attention, decoder bisa melihat kembali ke semua posisi input pada setiap langkah, sehingga tidak ada informasi yang terbuang meskipun kalimatnya panjang.

5. Berikan analogi sederhana untuk menjelaskan konsep Query, Key, Value kepada orang awam.

   Jawaban:

   - Query = pertanyaan yang sedang dicari → "apa yang saya butuhkan sekarang?"
   - Key = label setiap dokumen → judul berkas di rak
   - Value = isi dokumen sebenarnya → konten yang dibaca
   - Decoder bertanya → mencocokkan dengan label → mengambil isi yang paling cocok.

F. Pertanyaan Diskusi

1. Apa perbedaan antara Bahdanau attention dan Scaled Dot-Product Attention?

   Jawaban:

   Bahdanau menggunakan jaringan lapisan tersembunyi tambahan untuk menghitung skor (additive), cocok saat dimensi berbeda. Scaled Dot-Product menghitung kesamaan langsung dengan perkalian matriks lalu dibagi akar dimensi — lebih cepat, matematis lebih stabil, dan dipakai di Transformer.

2. Mengapa self-attention lebih baik dari recurrent connection untuk menangkap konteks jangka panjang?

   Jawaban:

   Self-attention menghubungkan semua kata dengan semua kata lain secara langsung, tidak bergantung urutan. Jarak antara kata awal dan akhir menjadi satu langkah saja, sedangkan di RNN harus melewati semua kata di antaranya. Informasi tidak hilang sepanjang rantai pemrosesan.

3. Apa yang dimaksud dengan multi-head attention? Mengapa menggunakan beberapa head?

   Jawaban:

   Memproses beberapa pasang QKV secara bersamaan dengan proyeksi berbeda. Setiap "kepala" mempelajari hubungan yang berbeda, satu kepala fokus pada ketergantungan posisi, yang lain pada makna semantik, dll. Hasilnya digabung → pemahaman lebih lengkap.

4. Mengapa Transformer (yang menggunakan self-attention) dapat diparalelkan, sedangkan RNN tidak?

   Jawaban:

   RNN harus menunggu langkah t-1 selesai sebelum langkah t dimulai, ketergantungan berurutan. Transformer menghitung semua posisi

sekaligus dalam satu operasi matriks — tidak ada ketergantungan antar posisi, sehingga bisa dibagi ke banyak prosesor sekaligus.

5. Bagaimana attention weights dapat digunakan untuk interpretabilitas model NLP?

   Jawaban:

   Bobot attention menunjukkan kata mana yang paling berpengaruh pada prediksi tertentu. Jika model salah, kita bisa melihat kata mana yang dijadikan fokus utama, apakah fokus pada kata yang benar atau justru kata yang tidak relevan. Membantu memahami alasan keputusan model dibuat.

Nilai Praktikum	
Catatan	
Dosen Pengampu	
Asisten Praktikum	
TTD Approval Asisten Praktikum	

CEKLIST PENIALAIAN PRAKTIKUM

No.	**Modul**	**NILAI**	**Catatan**
1	Modul 1 sesuaikan dengan CPMK-1		
2	Modul 2 sesuaikan dengan CPMK-2		
3			

Tanggal:

Praktikan: Layni Marlin	Dosen Pengampu: Ahmad Ashril Rizal, M.Cs.